University of Salerno

Department of Information and Electric Engineering
and Applied Mathematics

Master's Degree in Computer Engineering

A Semantic Parsing Approach to Knowledge Graph Question Answering over DBLP using LLMs and RAG

Supervisors:

Prof.ssa Sabrina Senatore

Prof. Nicola Capuano

Candidate:

Luca Gerbasi

S.N. 0622702125

Academic Year 2025/2026

Contents

1 Introduction

1.1 Problem Definition

Researchers and students regularly consult bibliographic databases of scientific publications, for example to find what has been published on a topic or to explore a new field of research. In the field of computer science, the dblp computer science bibliography is one of the most complete reference resources of this kind: as of 2024, it indexes over 7.2 million publications written by more than 3.5 million authors. The dblp team has designed an ontology to model the data (the dblp schema) and used it to build a Knowledge Graph (KG), called the dblp KG, that can be accessed through a public SPARQL endpoint [1].

While the SPARQL endpoint allows any user to extract information from the dblp KG, writing a SPARQL query is not a skill that most users have. It requires both knowledge of the query language and a precise understanding of the schema of the target KG. As a result, most users cannot access the information stored in the dblp KG, even though they could benefit from it.

Knowledge Graph Question Answering (KGQA) is the task of answering natural-language questions using the information stored in a Knowledge Graph [2]. A KGQA system takes a natural-language question and returns the corresponding answer from the KG. Different approaches have been proposed for this task. Most of the systems built for the dblp KG follow the semantic parsing approach: the question is first translated into a SPARQL query, and the query is then executed on the endpoint to retrieve the answer. These systems range from earlier ones that fine-tune a pre-trained language model on a dataset of question-SPARQL pairs to more recent ones that prompt a Large Language Model (LLM) without task-specific training.

The recent LLM-based systems still leave three open issues. First, several prompting techniques that have improved LLMs on complex reasoning tasks have not yet been explored for scholarly KGQA, and the contribution of the prompting techniques already used has not been measured in a controlled way. Second, none of these systems performs an explicit relation linking step that selects only the dblp ontology relations relevant to the user question; the full ontology is included in every prompt instead. Third, no work has tested whether scholarly KGQA can run on

consumer hardware, that is, whether a small, quantized LLM can produce SPARQL queries of competitive quality on this target.

In this thesis, we design and evaluate a KGQA system for the dblp KG that addresses these three issues. The proposed system follows the semantic parsing approach, is organized as a five-stage pipeline, and is evaluated on the DBLP-QuAD benchmark [3] with both a cloud LLM and a quantized model running on a consumer-hardware target.

1.2 Relevance of the Problem in the Context of Computer Science Engineering

Knowledge Graphs are now a common way to organize structured information in computer science. They are used in many areas of computer science, such as web search, biomedical informatics, enterprise data management, and digital libraries. The usefulness of a Knowledge Graph depends on how easily users can extract information from it. When the only way to access the data is a formal query language, this information stays available only to a small group of technical users.

Building natural-language interfaces to Knowledge Graphs is therefore a common problem in computer science engineering. It is not limited to a single application: any Knowledge Graph designed for a wide audience faces it. The problem combines several main areas of computer science. Natural-language processing is needed to understand the user question. Information retrieval is needed to find the parts of the Knowledge Graph that are relevant to the question. Software engineering is needed to combine these components into a reliable system.

The recent progress in Large Language Models has made this problem relevant again. Before LLMs, building a KGQA system required large annotated datasets and a fine-tuning step for every new Knowledge Graph. LLMs make this easier, but they raise new questions: how reliable the generated queries are, how much the prompt design affects the result, and whether the same techniques can run on consumer hardware. These questions are actively studied in computer science engineering. The dblp Knowledge Graph is a good example for studying them, because it is a reference resource for computer science itself, and the techniques used on it can also be applied to other domain-specific Knowledge Graphs.

1.3 Outline

We organize the rest of this thesis as follows. Chapter 2 presents the foundations of Knowledge Graphs, the reference ontologies in the bibliographic domain, and the state of the art in scholarly KGQA. Chapter 3 presents the original contribution: the proposed methodology, the design of the system, and the technologies used to implement it. Chapter 4 presents the experimental validation: the evaluation metrics, the experimental protocol, the dataset, the analysis of the results, and a final discussion of their significance and of possible directions for future work.

2 State of the Art

2.1 Semantics and Knowledge Graphs

The World Wide Web has changed the way people create, share, and access information. However, most web content is designed for humans: it uses formats like HTML that define how a page looks, but do not carry any meaning that machines can understand. This makes it difficult for software to automatically find, combine, and reason over the information available online. In 2001, Berners-Lee, Hendler, and Lassila proposed a new vision called the Semantic Web [4]. The idea is to extend the existing web by adding structured, machine-readable metadata to web resources, so that both computers and humans can use them effectively. The Semantic Web is not a separate web, but an enhancement of the current one.

This vision is based on a layered architecture known as the Semantic Web Layer Cake. At the base of this architecture lies the Resource Description Framework (RDF), a standard model for representing structured data. On top of RDF, languages such as RDF Schema (RDFS) and the Web Ontology Language (OWL) allow us to define the meaning of concepts and relationships within a specific domain. The SPARQL Protocol and RDF Query Language (SPARQL) provides a way to query RDF data. Finally, the Linked Open Data (LOD) principles describe how to publish and connect data on the web at a global scale. Together, these technologies are the foundation of Knowledge Graphs (KGs), which are large collections of structured knowledge used in both research and industry. In the following subsections, we describe each of these components, starting from the RDF data model and ending with the concept of Knowledge Graphs.

2.1.1 Resource Description Framework

The Resource Description Framework (RDF) [5] is the basic data model of the Semantic Web, defined by the World Wide Web Consortium (W3C). RDF allows us to represent information about web resources as structured statements. The basic unit in RDF is the triple, which has three parts: a subject, a predicate, and an object. The subject is the resource we are describing, the predicate is a property/relationship, and the object is the value that can be another resource or

a simple value like a string or a number. For example, the triple `dblp:pid/b/TimBernersLee, foaf:name, "Tim Berners-Lee"` states that the person identified by a certain DBLP URI has the name “Tim Berners-Lee”.

Resources and properties in RDF are identified by Uniform Resource Identifiers (URIs), which serve as unique global names (so data and metadata are treated in the same way). The main advantage of using URIs is that RDF data from different sources can be merged without conflicts. Besides URIs, RDF uses literals for concrete values such as strings and numbers, and blank nodes for resources that do not need a global name.

RDF can be written in different formats. RDF/XML was the first format defined by the W3C, but its syntax is complex and hard to read. For this reason, Turtle (Terse RDF Triple Language) is now the most common format because of its compact and readable syntax.

An RDF dataset can be seen as a directed graph, where subjects and objects are nodes and predicates are the labeled edges between them. This graph structure makes RDF a natural choice as the data model for Knowledge Graphs.

2.1.2 RDF Schema

RDF provides a way to make statements about resources, but it does not define the meaning of the terms used in those statements. RDF Schema (RDFS) [6] fills this gap by adding constructs like `rdfs:Class`, `rdfs:subClassOf`, `rdf:Property`, `rdfs:subPropertyOf`, `rdfs:domain`, and `rdfs:range`, which allow us to define type hierarchies and constraints on properties.
An important feature of RDFS is that it supports implicit knowledge. When we define class and property hierarchies, a reasoner (a tool that performs logical inference) can derive new facts that are not stated directly in the data. This ability to infer new knowledge automatically is one of the main advantages of Semantic Web technologies over traditional approaches, and it is important for answering questions over Knowledge Graphs.

2.1.3 Ontologies and OWL

RDFS allows us to define basic vocabularies, but many domains need more expressive power to represent complex relationships and rules. Ontologies address this need by providing formal

descriptions of the concepts, properties, and rules of a domain.

The most widely used definition describes an ontology as *"a formal, explicit specification of a shared conceptualisation."* [7]. This formulation was coined by Studer and colleagues, who merged Gruber's initial definition *("an explicit specification of a conceptualisation in an universe of discourse"* [8]) with the definition later given by Borst (*"formal specification of a shared conceptualization."* [9]). Each term in this formulation carries a specific meaning:

- Explicit: the types of concepts used and the restrictions must be clearly defined.
- Formal: it must be machine-processable.
- Shared: an ontology captures a consensual knowledge, not restricted to some individual.
- Conceptualization: an abstract and simplified model that identifies the relevant concepts of the world that we want to represent.

An ontology usually contains: concepts (or classes) that represent categories of things, relationships that connect concepts, attributes that describe individual entities, constraints that define rules about how concepts can be combined, and instances that represent specific entities. An ontology together with its instances forms a Knowledge Base (KB).

To represent such an ontology on the Semantic Web, the W3C extended RDFS with the Web Ontology Language (OWL) [10]. OWL comes in three versions of increasing complexity but OWL DL is the most widely used in practice. Based on a family of formal languages named Description Logics (DL), it applies specific restrictions to RDFS to ensure that reasoning remains computationally feasible.

2.1.4 SPARQL

SPARQL (SPARQL Protocol and RDF Query Language [11]) is the standard query language for RDF data, defined by the W3C. It allows for the creation of queries that describe sets of triple patterns to be matched within the data. A common SPARQL query includes a `SELECT` clause to list the variables to return and a `WHERE` clause to define the matching pattern. Variables, identified by the `?` prefix, are populated with matching values during execution.

In general, there are four types of SPARQL queries: `SELECT` returns matching values, `CONSTRUCT` creates new RDF data, `ASK` provides a boolean (yes/no) answer, and `DESCRIBE` retrieves infor-

mation about a specific resource. Furthermore, the protocol supports advanced features such as `FILTER`, `OPTIONAL`, aggregations (`COUNT`, `SUM`, `AVG`), `GROUP BY`, and `ORDER BY`."

SPARQL queries are sent to SPARQL endpoints, which are web services that accept queries over HTTP and return results in formats like XML or JSON. For example, the DBLP Knowledge Graph offers a public SPARQL endpoint [1] that gives access to its bibliographic data.

2.1.5 Linked Open Data

The Linked Open Data (LOD) initiative is the practical application of the Semantic Web vision at a global level. In 2006, Berners-Lee defined four principles for publishing structured data on the web [12]:

1. Use URIs as names for things.
2. Use HTTP URIs so that people can look up those names.
3. When someone looks up a URI, provide useful information using standards like RDF and SPARQL.
4. Include links to other URIs, so that users can discover more things.

These rules ensure that published data is not isolated, but part of a connected global space. To encourage openness, Berners-Lee also proposed a 5-star rating for open data: one star means data is available online under an open license; two stars means it is structured; three stars means it uses open formats; four stars means it uses RDF and URIs; and five stars means it links to other datasets.

The LOD Cloud, which shows all datasets published as Linked Data, has grown from 12 datasets in 2007 to more than 1,200, covering areas like government, science, and publishing. In the scholarly domain, DBLP is one of the most important datasets. Its RDF version and SPARQL endpoint make it a key resource in the Linked Open Data ecosystem [1], providing access to over 7 million publications. The DBLP data reuses standard vocabularies like FOAF, and the SPAR Ontologies [13], following the Linked Data principle of vocabulary reuse.

2.1.6 Knowledge Graphs

All the technologies described so far come together in the concept of Knowledge Graphs (KGs) that can be defined as a graph-based knowledge base that stores information about real-world entities and the relationships between them, usually using Semantic Web standards [14].

Knowledge Graphs have several advantages over traditional databases. First, their graph structure naturally represents complex relationships that are hard to model in relational tables. Second, the use of ontologies enables automated reasoning, so new facts can be derived from existing data. Third, the use of shared vocabularies and global identifiers makes it easy to combine data from different sources.

In the bibliographic domain, Knowledge Graphs like DBLP offer structured access to information about publications, authors, and venues. Being able to query these Knowledge Graphs is valuable for researchers and information systems. However, writing queries is difficult for non-expert users. This problem motivates the development of Knowledge Graph Question Answering(KGQA) systems, which take natural language questions and automatically translate them into formal queries. Building such a system for the DBLP Knowledge Graph is the main goal of this thesis, and we discuss the current state of research on KGQA in the following sections.

2.2 Reference Ontologies in the Bibliographic Domain

In the bibliographic domain, several ontologies have been developed to describe publications, authors, citations, and venues using Semantic Web technologies and Knowledge Graphs. In the following subsections, we review three relevant approaches: the Bibliographic Ontology (BIBO), the Semantic Publishing and Referencing (SPAR) Ontologies, and the dblp ontology. We highlight the points of contact and the design differences among them.

2.2.1 The Bibliographic Ontology

The Bibliographic Ontology, commonly known as BIBO, is a vocabulary for describing bibliographic resources on the Semantic Web using RDF [15]. BIBO was designed as a complement to the Dublin Core metadata terms [16], which provide general properties like title, creator, and date.

BIBO extends these terms with more specific classes and properties for bibliographic description. It can be used as a citation ontology, as a document classification ontology, or simply as a way to describe any kind of document in RDF.

The core of BIBO is a class hierarchy rooted in `bibo:Document`, which represents any bounded body of information designed to communicate. This class has several subclasses for specific document types, such as `bibo:Article`, `bibo:AcademicArticle`, `bibo:Book`, `bibo:Proceedings`, `bibo:Thesis`, `bibo:Report`, and `bibo:Chapter`, among others. BIBO also defines properties for common bibliographic identifiers like `bibo:doi` and `bibo:isbn`, as well as properties for pagination, volume, and issue numbers. An important characteristic of BIBO is that it uses a flat document model: each bibliographic resource is described as a single entity, without distinguishing between the abstract intellectual content and its physical or digital manifestation. This is a simple and practical approach, but it limits the ability to represent the different levels of abstraction that characterize a bibliographic work. A more expressive approach to this problem is provided by the SPAR Ontologies.

2.2.2 The SPAR Ontologies

The Semantic Publishing and Referencing (SPAR) Ontologies are a suite of complementary ontologies for the description of the main areas of the scholarly publishing domain [13]. Unlike BIBO, which is a single ontology, SPAR follows a modular design philosophy: each ontology in the suite addresses a specific aspect of scholarly publishing, and they can be used independently or combined together.

The most important SPAR ontologies are: FaBiO (FRBR-aligned Bibliographic Ontology), for bibliographic entity description; CiTO (Citation Typing Ontology), for the characterization of citations; DataCite Ontology for metadata about persistent identifiers; BiRO (Bibliographic Reference Ontology), for modeling bibliographic references and reference lists; PRO (Publishing Roles Ontology), for defining roles in the publishing process; PSO (Publishing Status Ontology), for tracking publication statuses; DoCO (Document Components Ontology), for describing the structural parts of documents; and C4O (Citation Counting and Context Characterization Ontology), for citation contexts and counts. Among all these ontologies, FaBiO, CiTO, and the DataCite Ontology are the most relevant in the context of this thesis, because they address

three important aspects of bibliographic Knowledge Graphs: the description of publications, the characterization of citation relationships, and the management of persistent identifiers.

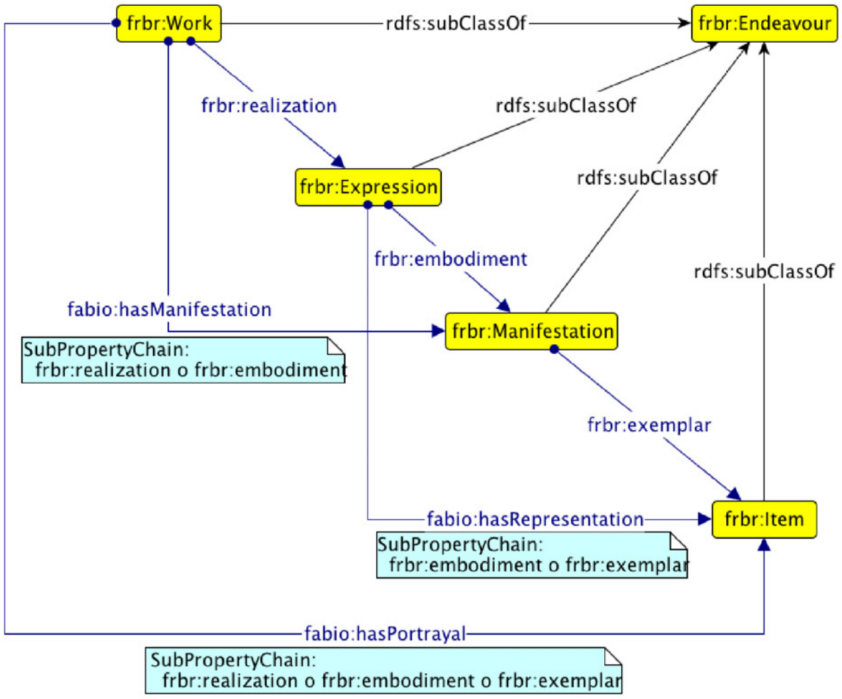

Figure 2.1: *The main FRBR object properties and the related properties introduced by FaBiO*

FaBiO, the FRBR-aligned Bibliographic Ontology, provides classes for describing bibliographic entities following the Functional Requirements for Bibliographic Records (FRBR) model [17]. The FRBR model organizes bibliographic entities into four hierarchical layers. The first layer is the Work, which represents the abstract intellectual or artistic creation (for example, the idea of a research article). The second layer is the Expression, which represents a specific realization of a work (for example, the author's manuscript or the published version). The third layer is the Manifestation, which represents the physical or digital embodiment of an expression (for example, a PDF file or a printed copy). The fourth and final layer is the Item, which represents an individual copy of a manifestation. FaBiO maps its bibliographic classes to these layers: for example, a `fabio:JournalArticle` is modeled as a subclass of `frbr:Expression`, while its printed or digital forms are described as manifestations. This multi-layer approach is more expressive than the flat model of BIBO, because it allows us to distinguish between the intellectual content of a work and its various realizations, but it also introduces additional complexity.

CiTO, the Citation Typing Ontology, provides a vocabulary for characterizing the nature of citations between scholarly works [17]. In most bibliographic systems, a citation is represented as a simple link from one publication to another, without any information about the reason for

the citation. CiTO addresses this limitation by defining typed citation relationships that capture the rhetorical function of a citation. For example, `cito:citesAsAuthority` indicates that a work is cited as an authoritative source, `cito:extends` indicates that the citing work builds upon the cited work, `cito:critiques` indicates a critical evaluation, and `cito:usesMethodIn` indicates that the citing work adopts a method described in the cited work. CiTO defines more than 60 such properties.

The DataCite Ontology provides a standardized vocabulary for representing and managing the various external identifiers associated with scholarly entities [18]. Its central class is `datacite:Identifier`, which represents any external identifier associated with a bibliographic entity. Identifier entities are linked to the entity they identify via the `datacite:hasIdentifier` property, and to their identifier scheme via `datacite:usesIdentifierScheme`. The ontology defines a hierarchy of identifier types, including `datacite:ResourceIdentifier` for resources such as datasets and publications, and `datacite:PersonalIdentifier`, `datacite:OrganizationIdentifier`, and `datacite:FunderIdentifier` for different kinds of agents. This approach allows a single, uniform mechanism to represent identifiers of many different schemes, such as DOI, ORCID, arXiv, ISBN, and ISSN. As we will see, the dblp Knowledge Graph directly reuses the DataCite Ontology for modeling its external identifiers.

One of the most important practical applications of the SPAR Ontologies is OpenCitations, an infrastructure organization for open scholarship [19]. OpenCitations publishes open citation data as Linked Open Data using SPAR ontologies: it uses FaBiO for describing bibliographic entities and CiTO for characterizing citation relationships. Its largest dataset is COCI (the OpenCitations Index of Crossref Open DOI-to-DOI Citations), which contains over 624 million bibliographic citations. OpenCitations assigns each publication a unique identifier called OMID (OpenCitations Meta Identifier), which is used by other Knowledge Graphs, including dblp, to establish links to the OpenCitations data. However, since OpenCitations relies on openly available metadata, its coverage is limited to publications with open citation data. In contrast, dblp provides more complete bibliographic coverage for computer science because it also indexes publications whose metadata is not openly available, making it a more comprehensive resource for building a KGQA system in the scholarly domain.

2.2.3 The dblp Knowledge Graph

The dblp computer science bibliography is one of the most comprehensive reference resources for bibliographic metadata in computer science. It has been maintained for more than 30 years, and as of 2024, it indexes over 7.2 million publications by more than 3.5 million authors [1]. While the semantic content of dblp had been published as RDF by third parties in the past, most of these external resources disappeared from the web or became outdated over time. The dblp Knowledge Graph (dblp KG) is the first official semantic representation of the dblp data, designed and maintained by the dblp team at Schloss Dagstuhl. Unlike previous efforts, the dblp KG is continuously synchronized with the live dblp data, ensuring that it always reflects the current state of the bibliography.

A key design decision of the dblp KG is that it does not reuse existing bibliographic ontologies like BIBO or FaBiO directly. The dblp ontology was designed to model the data the way dblp handles and curates it, including the specific characteristics and peculiarities of its approach. There are several reasons for this choice. First, the dblp data model is incompatible with the FRBR model: dblp records do not distinguish between the Work and Expression layers, and do not address the Manifestation or Item layers at all. Second, the dblp publication type system was originally derived from BibTeX entry types, which do not map cleanly to the classes defined in BIBO or FaBiO. Third, the dblp approach to author disambiguation uses pseudo-author entities that have no counterpart in standard bibliographic ontologies. Nevertheless, the dblp ontology provides links to related types and predicates from existing ontologies whenever possible, following the Linked Data principle of vocabulary reuse.

The dblp ontology defines three core entity types, all subclasses of an abstract `dblp:Entity` supertype: Publication, Creator, and Stream. Each type has its own IRI namespace: publications use `https://dblp.org/rec/`, creators use `https://dblp.org/pid/`, and streams use `https://dblp.org/streams/`. The relationships between these types form the backbone of the Knowledge Graph. Publications are linked to their creators via the `dblp:createdBy` predicate and to their publication venues via `dblp:publishedInStream`.

The `dblp:Publication` class represents any academic work indexed in dblp, and it is specialized into several subtypes that reflect the categories of publications. The most common types are `dblp:Inproceedings` for conference and workshop papers, `dblp:Article` for journal articles,

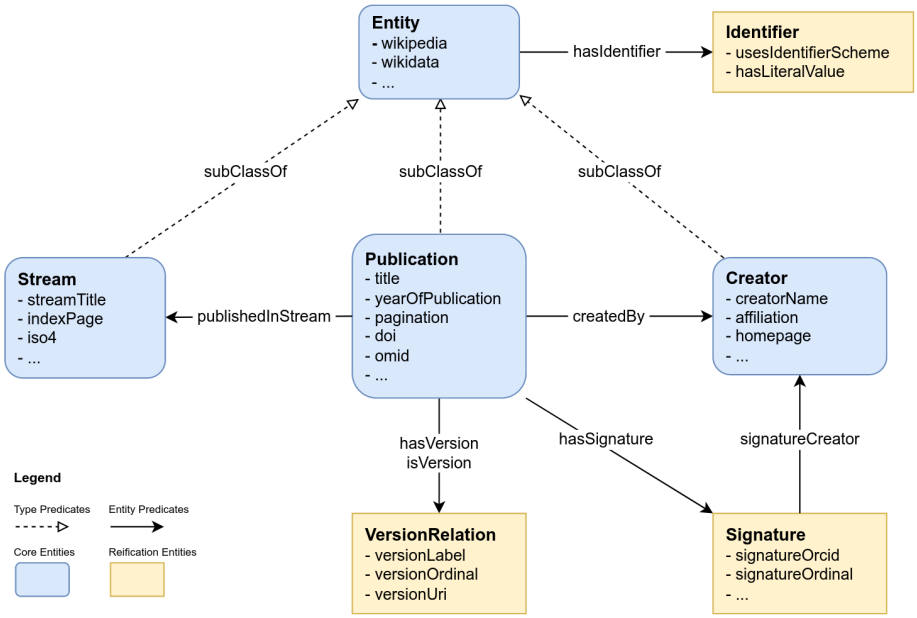

Figure 2.2: *Excerpt from the dblp ontology showing the relationships between core and reification types. Each of the entity boxes shows the type at the top, followed by a list of predicates of entities of that type. The figure shows only a small selection of all predicates and omits most of the finer-grained subtypes.*

and `dblp:Informal` for preprints and non-peer-reviewed works. Key properties of publications include `dblp:title`, `dblp:yearOfPublication`, and `dblp:pagination`.

The `dblp:Creator` class represents any individual or group listed as author or editor of a publication. It has three subtypes: `dblp:Person` for individual persons, `dblp:Group` for organizations and collaborations, and `dblp:AmbiguousCreator` for cases where the true authorship behind a name string is unknown. The `dblp:AmbiguousCreator` type is a feature unique to dblp: when the editorial team cannot determine which real person is behind a specific name, a pseudo-author entity is created to collect all unresolved publications under that name. This entity is volatile and changes as the dblp team resolves disambiguation cases over time. Creator metadata includes properties such as `dblp:creatorName`, `dblp:affiliation`, and `dblp:homepage`.

The `dblp:Stream` class represents any regular source of publications, such as a journal, a conference series, a book series, or a repository. Stream entities were introduced in June 2024 as a major extension to the dblp KG schema, which added publication venues as first-class entities in the Knowledge Graph. The class has four subtypes: `dblp:Journal` for periodically published journals, `dblp:Conference` for conference and workshop series, `dblp:Series` for book series, and `dblp:Repository` for sources of research data like Zenodo. Stream entities carry metadata

such as `dblp:streamTitle`, `dblp:issn`, and `dblp:wikidata` and they can be linked to each other through hierarchical predicates like `dblp:subStream` and `dblp:superStream`, as well as temporal predicates like `dblp:predecessorStream` and `dblp:successorStream`, which allow the representation of venue histories and relationships.

Besides the three core entity types, the dblp ontology defines three types of reification entities that are used to attach additional metadata to the relationships between core entities. Unlike core entities, reification entities are not assigned persistent IRIs and are represented as blank nodes in the RDF graph. The three types are Signature, Identifier, and VersionRelation.

The `dblp:Signature` class reifies the link between a publication and one of its creators. The dblp ontology represents this authorship relationship redundantly in two ways: a direct link using the `dblp:createdBy` predicate, and a more detailed link through `dblp:hasSignature` that points to a Signature entity. This redundancy is intentional: the direct predicate allows simple and convenient queries when only the link between a publication and its creators is needed, while the Signature entity provides access to additional metadata about the authorship. Specifically, each Signature records the relative position of the creator in the author list via `dblp:signatureOrdinal` and may link to an ORCID IRI stated in the publication metadata via `dblp:signatureOrcid`. To distinguish between different roles, the class is further specialized into `dblp:AuthorSignature` and `dblp:EditorSignature`.

The `datacite:Identifier` class, reused from the DataCite Ontology described earlier, is used to annotate core entities with external identifiers such as DOIs, ORCIDs, and Wikidata IRIs. Each Identifier entity is linked to a core entity via `datacite:hasIdentifier`, to its identifier scheme (for example, `datacite:doi` or `datacite:orcid`) via `datacite:usesIdentifierScheme`, and to its literal value via `litre:hasLiteralValue`. It is important to note that Identifier entities do not link directly to the IRIs of the external resources, in order to support a wider range of identifier schemes. In addition to the indirect identifier mechanism, the dblp ontology also provides direct IRI predicates for the most important external identifiers, following the same redundancy principle used for Signatures. For publications, these direct predicates are `dblp:doi`, `dblp:isbn`, `dblp:wikidata`, and `dblp:omid`. For creators, they are `dblp:orcid` and `dblp:wikidata`. For streams, they are `dblp:issn` and `dblp:wikidata`.

The `dblp:VersionRelation` class models hierarchies between publications, where one publication is a version (or instance) of another publication (called the concept). This entity type was

introduced together with the `dblp:Data` publication type to support cases where, for example, a research dataset has multiple released versions. Publications are linked to VersionRelation entities via the predicates `dblp:versionConcept` and `dblp:versionInstance`, and each relation can carry metadata such as a version label and a relative order among instances. For convenience, the direct predicate `dblp:isVersionOf` is also provided to link publications directly without going through the reification entity.

Access to the dblp KG is provided through several channels. The primary access point is a public SPARQL endpoint[1] powered by the QLever engine, which also provides a user interface. In addition, daily updated RDF dumps are available in multiple formats (RDF/XML, N-Triples, Turtle), and persistent monthly releases are archived for reproducibility. A Linked Open Data API allows individual pieces of RDF data to be retrieved for any creator, publication, or stream by appending a file extension to its IRI. The dblp team provides also instructions for setting up a self-hosted replica of the SPARQL endpoint with the same functionality.

The dblp SPARQL endpoint also supports federated queries via the SPARQL `SERVICE` keyword, which allows combining data from the dblp endpoint with data from other SPARQL endpoints in a single query. For example, a federated query can retrieve author information from dblp and combine it with biographical data from Wikidata [20], such as the place of birth or geographic coordinates. While citation data from OpenCitations could also be accessed via federated queries, the dblp team chose to filter the OpenCitations dataset to include only citations involving dblp-indexed publications and to integrate the result directly into the combined graph. This avoids the performance overhead of federated queries for citation lookups and ensures that citation data is always available locally.

As it is evident from the ontology just presented, and as discussed in the previous section, writing correct SPARQL queries requires knowledge of both the query language and the structure of the target Knowledge Graph. This motivates the development of Knowledge Graph Question Answering (KGQA) systems, which we discuss in the following section.

[1] https://sparql.dblp.org/sparql

2.3 Knowledge Graph Question Answering (KGQA) Systems

Knowledge Graph Question Answering (KGQA) is the task of finding answers to questions posed in natural language, using the information stored in a Knowledge Graph[2]. A KGQA system receives a natural language question as input and returns an answer by accessing the KG. The main challenge lies in the fact that natural language is inherently ambiguous and flexible, while the underlying data is structured and requires precise access patterns.

The approaches used in KGQA can be grouped into two main categories [21]. The first is the semantic parsing approach, which translates the natural language question into an executable formal query, such as a SPARQL query. The generated query is then executed against the KG endpoint to retrieve the answer. The second is the information retrieval approach, also called the retriever-reasoner framework, which retrieves relevant subgraphs from the KG and then applies reasoning to extract the answer directly from the retrieved data, without generating a formal query.

Semantic parsing produces interpretable and verifiable queries, which makes it easier to understand how the system arrived at an answer. However, it requires the system to generate syntactically and semantically correct queries, which is a challenging task. The information retrieval approach avoids the need for query generation, but it typically requires direct access to the graph structure for subgraph retrieval, which for large Knowledge Graphs such as dblp may pose significant storage and hardware requirements. In the context of this thesis, we adopt the semantic parsing approach, which allows us to query the dblp KG through its public SPARQL endpoint without the need to store the graph locally.

2.3.1 The Semantic Parsing Pipeline

A typical semantic parsing KGQA system follows a pipeline architecture with three main subtasks: entity linking, relation linking, and query generation [2]. Each sub-task addresses a specific aspect of the translation from natural language to formal query, and the output of each step is passed to the next one in the pipeline.

Entity linking is the first step. It involves detecting mentions of named entities in the natural language question and mapping them to the corresponding resources in the Knowledge Graph.

For example, given the question “Which papers did Tim Berners-Lee publish?”, the entity linker must recognize “Tim Berners-Lee” as a person entity and map it to the correct URI in the KG (in this case, `https://dblp.org/pid/b/TimBernersLee`). This task is challenging because entity names in natural language can be ambiguous, abbreviated, or written in different forms than the labels stored in the KG.

Relation linking is the process of identifying the KG predicates that are relevant to the question. Given the same example, the relation linker must determine that the `dblp:createdBy` predicate connects publications to their authors. This step requires understanding both the meaning of the question and the schema of the target KG. In some systems, relation linking is performed explicitly as a separate module, while in others it is handled implicitly by the query generator.

Query generation is the final step, where the linked entities and relations are combined with SPARQL vocabulary to produce a correct and executable query. This step must determine the correct structure of the query, including the query form (SELECT, ASK, COUNT), the arrangement of triple patterns, and the use of modifiers such as filters, negation, or aggregation. Query generation has been approached in different ways, from template-based methods and intermediate graph representations to neural models that generate SPARQL queries token by token [2]. We discuss the specific systems proposed for scholarly KGQA and their query generation strategies in the Detailed Analysis section.

2.3.2 The Role of Large Language Models in KGQA

The introduction of Large Language Models (LLMs) has changed the way KGQA systems approach the query generation task. Earlier systems relied on fine-tuning pre-trained language models on datasets of question-SPARQL pairs [2]. While this approach achieved strong results, it requires a large amount of training data and a separate fine-tuning step for each new KG or domain. LLMs offer an alternative: they can generate SPARQL queries through prompting strategies, without the need for task-specific fine-tuning [21]. In the following paragraphs, we first describe the prompting techniques that have already been applied to scholarly KGQA, and then introduce additional general-purpose LLM techniques that are relevant to the system presented in this thesis.

Few-shot prompting is one of the most effective strategies for using LLMs in KGQA [22]. In this

approach, a small number of question-SPARQL pairs are included in the prompt as examples, together with the test question. The LLM learns from these examples and generates a SPARQL query for the new question. The quality of the generated query depends strongly on the relevance of the examples: providing examples that are similar to the test question leads to better results than using random or generic examples [21]. A related distinction is between static few-shot, where a fixed set of examples is used for all questions, and dynamic few-shot, where the examples are selected based on the specific question.

Retrieval-Augmented Generation (RAG) is a technique that augments the knowledge available to an LLM by retrieving relevant external data and including it in the prompt at inference time [23]. The general idea is that LLMs are limited to the knowledge acquired during training, and RAG allows them to access additional information without the need for retraining. A typical RAG system works in two phases: an indexing phase, where the source data is encoded into numerical vectors called embeddings and stored in a vector database, and a retrieval phase, where the system computes the embedding of the user query and searches for the most similar entries using a distance metric such as cosine similarity. The retrieved data is then included in the prompt alongside the query, providing the LLM with relevant context for generating its response.

In the context of KGQA, RAG is used to dynamically select the most relevant few-shot examples for the prompt. Instead of using a fixed set of static examples, the training questions are encoded using an embedding model and stored in a vector database. At inference time, the system computes the embedding of the test question, retrieves the top-k most similar training questions, and includes them in the prompt together with their corresponding SPARQL queries [21]. This dynamic example selection ensures that the LLM receives examples that are semantically close to the question it needs to answer, which improves the accuracy of the generated queries compared to static few-shot prompting.

Beyond the techniques already adopted in scholarly KGQA, several general-purpose LLM techniques are relevant to the system we present in this thesis.

Chain-of-Thought (CoT) prompting is a technique that elicits step-by-step reasoning from LLMs before they produce a final answer [24]. The core idea is that, by generating intermediate reasoning steps, the model is forced to decompose a complex problem into smaller logical parts, which leads to more accurate results. CoT can be applied in a few-shot setting, where each example in the prompt includes not only the question and the answer but also the reasoning steps that

connect them, so that the model learns to produce its own reasoning chain. It can also be applied in a zero-shot setting, by simply instructing the model to reason step by step without providing any examples.

Finally, structured output is a technique for constraining the format of the LLM response. Instead of generating free-form text, the LLM is instructed to produce its output according to a predefined schema, for example a JSON object with specific fields for the generated query and an optional explanation. This ensures that the response can be parsed programmatically and reduces the risk of malformed or incomplete outputs. Modern LLM frameworks support structured output through mechanisms such as JSON schemas or data validation libraries.

2.3.3 Benchmarks and Datasets

The evaluation of KGQA systems relies on benchmark datasets that provide pairs of natural language questions and their corresponding formal queries. These benchmarks allow researchers to measure the accuracy of a system by comparing the answers produced by the generated queries against the expected results. In the general KGQA domain, several benchmarks have been proposed over the years, targeting encyclopedic Knowledge Graphs such as DBpedia and Wikidata. Among the most established is the QALD (Question Answering over Linked Data) challenge series [25].

In the scholarly domain, the Scholarly QALD challenge[2], held at the International Semantic Web Conference (ISWC), provides a dedicated evaluation framework for question answering over academic Knowledge Graphs. The challenge includes two tasks targeting two different KGs. The first task is based on SciQA [26], a benchmark of manually crafted and automatically generated questions over the Open Research Knowledge Graph (ORKG). The second task uses DBLP-QuAD [3], the largest scholarly KGQA dataset, consisting of 10,000 question-SPARQL pairs over the DBLP Knowledge Graph. DBLP-QuAD covers 10 different query types, including single fact, multi-fact, boolean, negation, count, and superlative questions, and provides a standardized split for training, validation, and testing. DBLP-QuAD is the dataset used in this thesis for evaluating our system, and we describe it in detail in chapter 4.

[2]https://kgqa.github.io/scholarly-QALD-challenge/

2.4 Detailed Analysis of the State of the Art

In this section, we examine the architectures of the three most relevant KGQA systems proposed for the DBLP Knowledge Graph. We present them in chronological order, from the earliest fine-tuning-based baselines to the most recent LLM-based approach.

2.4.1 The DBLP-QuAD Baseline

The first KGQA system for the DBLP Knowledge Graph was introduced together with the DBLP-QuAD dataset by Banerjee et al.[3]. This baseline follows the approach proposed in an earlier work by the same authors[2], where pre-trained language models are fine-tuned for the task of SPARQL query generation. The system assumes that entity linking and relation linking have already been performed, and it focuses exclusively on the query building step: given a natural language question and the gold (correct) entity URIs and relation URIs, the task is to arrange them in the correct order together with SPARQL vocabulary tokens to produce a valid query.

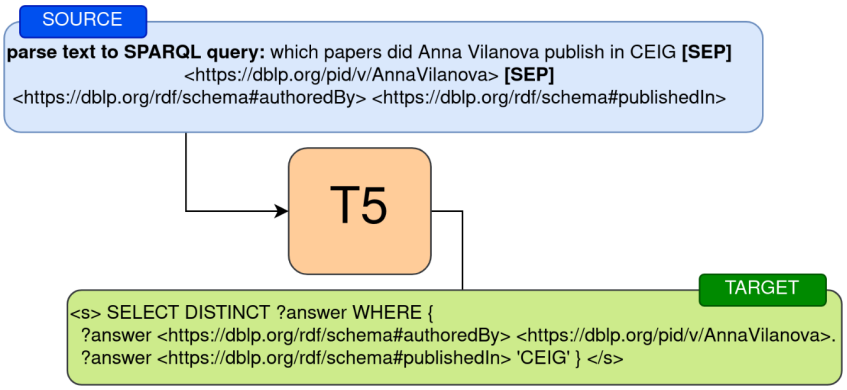

Figure 2.3: *Representation of source and target text used to fine-tune the T5 model*

The baseline uses T5, a Text-to-Text Transfer Transformer that casts every NLP task as a text-to-text problem. The input to the model is formed by concatenating the natural language question with the gold entity URIs and gold relation URIs, separated by a special `[SEP]` token. Since the T5 tokenizer splits URLs into arbitrary sub-word fragments, which the model then fails to reconstruct correctly in the output, the authors use the sentinel tokens provided by T5 to represent DBLP URI prefixes. For example, the prefix `https://dblp.org/rdf/schema#` is mapped to a specific sentinel token like `<extra_id_1>`. The same approach is applied to SPARQL vocabulary

keywords. The output of the model is the complete SPARQL query, generated token by token, where entity and relation URIs appear as a sentinel token prefix followed by the entity or relation identifier.

Two model variants are evaluated: T5-Small (60 million parameters) and T5-Base (220 million parameters). Both models are fine-tuned on the DBLP-QuAD training set (7,000 question-query pairs). On the DBLP-QuAD test set (2,000 questions), T5-Base achieves an F1 score of 0.868 and T5-Small achieves 0.721. It is important to note that these results are obtained in a gold setting, where the correct entity and relation URIs are provided as input. This means the system does not perform entity linking or relation linking, and therefore its results represent an upper bound on the query building component alone. Moreover, the fine-tuning approach requires a large training dataset and a full retraining cycle whenever the underlying KG schema changes.

2.4.2 NLQxform

NLQxform is a KGQA system developed by Wang et al. [27] for the Scholarly QALD Challenge at ISWC 2023, where it ranked first on the DBLP-QuAD question answering task with an F1 score of 0.8488 on test questions. Unlike the DBLP-QuAD baseline, NLQxform is a complete end-to-end system that includes entity linking and does not assume gold annotations at inference time. The system follows a four-step pipeline.

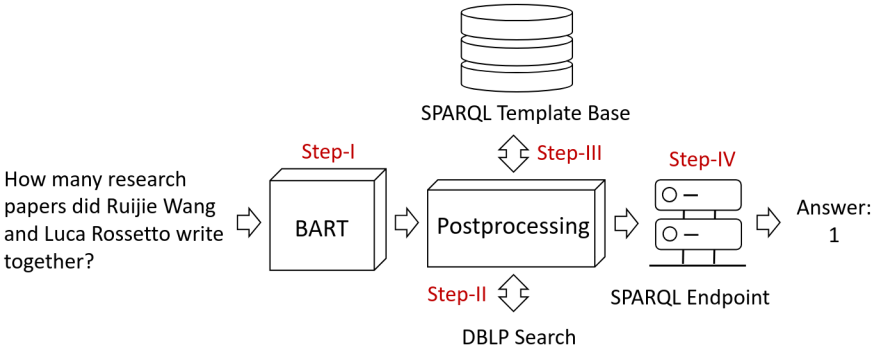

Figure 2.4: *An overview of the NLQxform system and its four steps question-answering process.*

In the first step, a BART (Bidirectional and Auto-Regressive Transformer) model is fine-tuned on the DBLP-QuAD training set to translate the natural language question into a logical form. BART is a pre-trained language model designed for text generation tasks such as translation and summarization, which takes an input text and produces an output text. This logical form has

the structure of a SPARQL query, but it retains entity mentions as natural language text instead of replacing them with URIs. To help the model learn the SPARQL syntax, the authors add all SPARQL syntax elements (clauses, functions, modifiers, punctuation) and all DBLP relation URIs as special tokens to the BART tokenizer. Adding the relations as special tokens allows the model to directly generate relation URIs in the logical form, effectively handling relation linking implicitly during the translation step.

In the second step, NLQxform performs entity linking using the DBLP search APIs. For each entity mention recognized in the logical form, the system queries the corresponding DBLP API endpoint (for example, the author search API for person entities) and retrieves a ranked list of candidate URIs.

In the third step, the system applies a template-based query correction mechanism. During training, the gold SPARQL queries are used to build a template base by replacing entity URIs with placeholder variables. At inference time, for each generated logical form, the system retrieves the top-3 most similar templates from this base using string similarity. This step is designed to correct minor syntax and grammar errors in the logical form produced by BART.

In the fourth and final step, the retrieved templates are filled with the entity URIs obtained from entity linking to generate candidate SPARQL queries. These candidates are executed against the DBLP SPARQL endpoint, and the first query that returns a non-empty result is selected as the final answer. When the entity linker returns multiple candidate URIs for a single mention, the system iterates through them in ranked order, selecting the first combination that produces a valid result.

NLQxform achieved an entity linking F1 of 0.7961 and a question answering F1 of 0.8488. However, the template-based correction step makes the system dependent on the templates observed during training: when the KG schema changes or new query patterns are introduced, both the BART model and the template base need to be updated.

2.4.3 ASK-DBLP

ASK-DBLP is the most recent KGQA system for the DBLP Knowledge Graph, proposed by Taffa et al. [28]. Unlike the previous systems that rely on fine-tuned models, ASK-DBLP adopts a prompting-based approach using a Large Language Model, specifically Qwen 2.5 Coder 32B

Instruct, without any task-specific fine-tuning. The system builds on the few-shot prompting strategy introduced by Taffa and Usbeck [21], who demonstrated that LLMs can generate SPARQL queries for scholarly KGs when provided with similar question-SPARQL pairs as in-context examples. ASK-DBLP extends this approach by incorporating entity linking, schema context, and a human-in-the-loop interaction layer. The overall pipeline operates in seven steps.

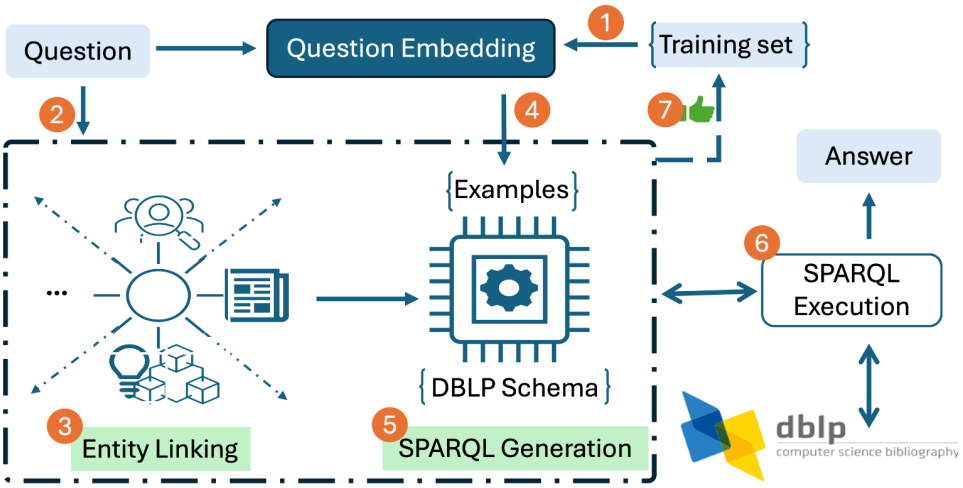

Figure 2.5: *ASK-DBLP pipeline. The system (1) embeds training questions offline. When a user enters a question, it (2) checks question clarity, (3) performs entity linking, and simultaneously (4) retrieves the most similar training questions. (5) The SPARQL generator constructs a prompt and queries the LLM to produce a SPARQL query. (6) The generated query is executed against the DBLP SPARQL endpoint. (7) If the user confirms the correctness of the query and answer, the question-SPARQL pair is added to the training set.*

In the first step, the system pre-computes embeddings of all training questions from the DBLP-QuAD dataset. This offline step enables efficient similarity-based retrieval at query time.

In the second step, a question clarity checker uses the LLM to assess whether the user question is complete and unambiguous. If a question is vague, for example “Give me the best database paper”, the system prompts the user to reformulate it with more specific criteria.

In the third step, entity linking is performed by DBLPLink 2.0 [29], a dedicated entity linker for the DBLP KG that follows a zero-shot approach that does not require any training data. The entity linking pipeline consists of four stages:

- In the first stage, an LLM (Qwen-2.5-3B) receives a prompt that instructs it to extract named entity mentions from the input question and classify each mention as person, publication, or venue.

- In the second stage, each extracted mention is searched in a text search index that contains the labels of all DBLP entities, organized by type. The search returns a ranked list of candidate entities for each mention.
- In the third stage, the system uses the KG to gather context about each candidate entity. For each candidate, the system retrieves the triples that are directly connected to it in the KG (its one-hop neighborhood) and converts them into readable sentences. For example, for the candidate “Ashish Vaswani”, these sentences might include “Ashish Vaswani - authored - Attention is All You Need” and “Ashish Vaswani - affiliated with - Google Brain”. This context helps the system understand who each candidate is and what they are connected to.
- In the fourth stage, the system uses the LLM to evaluate how well each candidate matches the original question. Each neighborhood sentence is presented to the LLM together with the question, and the LLM assigns a confidence score that indicates how consistent the candidate is with the context of the question. For each candidate, the scores from all its neighborhood sentences are averaged, and the candidate with the highest average score is selected as the final linked entity.

Simultaneously, in the fourth step, the system performs dynamic few-shot retrieval, which follows the RAG-based approach described earlier in this chapter. The system retrieves the top-5 most similar training questions from the DBLP-QuAD dataset using embedding-based cosine similarity, and includes them together with their corresponding SPARQL queries as in-context examples for the LLM.

In the fifth step, the system performs SPARQL generation. It constructs a prompt that includes the user question, the retrieved question-SPARQL examples, the linked entities with their types and URIs, and the DBLP ontology schema. The schema context covers essentially the entire DBLP ontology (except a few inverse-of properties not used in DBLP-QuAD), with each property listed together with its URI and natural language description. The prompt is passed to Qwen 2.5 Coder 32B Instruct, which generates a SPARQL query.

In the sixth step, the generated SPARQL query is executed against the DBLP SPARQL endpoint and the results are presented to the user. ASK-DBLP provides an interactive web interface where users can also select alternative entity candidates from the entity linker results and edit the generated SPARQL query before execution.

In the seventh step, the user can provide feedback on the generated query and answer. When a user confirms that a generated query and its answer are correct, the question-SPARQL pair is added to the training set, enabling the system to improve incrementally over time.

On the DBLP-QuAD test set (2,000 questions), ASK-DBLP achieves an F1 score of 0.7614. This is lower than the F1 scores of the DBLP-QuAD baseline (0.868) and NLQxform (0.8488), but the comparison is not straightforward. The DBLP-QuAD baseline operates in a gold entity and relation setting, where the correct URIs are provided as input, and NLQxform relies on fine-tuned models and templates built from the training data. ASK-DBLP is the only system that uses an LLM without any task-specific fine-tuning, generating SPARQL queries through in-context learning alone. This makes it more adaptable to changes in the DBLP KG schema without retraining, but it also explains the lower F1 score compared to the fine-tuned approaches.

2.5 Identification of Potential Advancements over the State of the Art

The analysis of existing systems reveals several areas where the current state of the art can be improved.

The first area concerns the exploration and evaluation of prompting techniques for SPARQL generation. ASK-DBLP and its precursor [21] demonstrated that few-shot prompting with similar question-SPARQL pairs enables LLMs to generate SPARQL queries without fine-tuning. However, neither of these works provides a controlled evaluation of the individual contribution of each component of the prompt. For example, it is not clear whether the dynamic retrieval of similar examples offers a measurable advantage over a static set of few-shot examples, or how much each part of the prompt contributes to the overall performance. Furthermore, techniques that have been shown to improve LLM performance on complex reasoning tasks have not been explored in the context of scholarly KGQA. Chain-of-Thought reasoning [24], where the model is asked to produce an intermediate explanation before the final answer, has not been applied. Similarly, structured output, where the LLM response is constrained to a predefined schema (for example, a JSON object with separate fields for a reasoning explanation and the SPARQL query), has not been investigated.

The second area concerns the approach to relation linking. As noted in the comparison of existing systems, none of them implements an explicit relation linking component that dynamically selects the most relevant ontology properties for a given question. The DBLP-QuAD baseline and NLQxform avoid the problem by relying on gold relations or implicit handling through fine-tuning. ASK-DBLP does not perform relation linking either: it includes the full DBLP ontology schema in every prompt regardless of the question content. While this approach works well for large models, it introduces noise when the prompt contains properties that are irrelevant to the current question. This is particularly problematic for smaller models with limited context capacity, where irrelevant properties can confuse the model and lead to incorrect predicate selection. A dynamic approach that selects only the relevant properties for each question could reduce this noise and improve accuracy.

The third area concerns the feasibility of running scholarly KGQA on consumer-grade hardware. ASK-DBLP uses Qwen 2.5 Coder 32B Instruct, a model whose size exceeds the memory capacity of the GPUs commonly available on consumer hardware. The question of whether a small, quantized LLM running on consumer hardware can achieve competitive performance on the DBLP-QuAD benchmark has not been investigated.

The methodology we propose to address these three areas is presented in chapter 3.

3 Original Contribution to the Problem Solution

3.1 Definition of the Proposed Methodology

We propose a scholarly KGQA system based on the semantic parsing approach introduced in chapter 2. Compared to the existing scholarly KGQA systems analyzed in that chapter, the proposed methodology introduces three contributions:

- A pipeline for SPARQL generation that integrates dynamic few-shot retrieval, query-type filtering, Chain-of-Thought reasoning, and structured output, accompanied by an ablation study that isolates the contribution of each technique to the quality of the generated SPARQL queries.
- An explicit relation linking module based on Retrieval-Augmented Generation that retrieves the ontology predicates most relevant to the user question, instead of including the full DBLP schema in every prompt.
- The evaluation of the same pipeline with both a cloud LLM and a 4B-parameter quantized model running on consumer hardware, to determine which prompting techniques are most critical for compensating the reduced model capacity.

The architecture of the system was designed to support these three contributions. Every module is decoupled from the services it relies on (the LLM, the embedding model, …) and admits more than one implementation. Every choice that defines an experimental run, namely the implementation of each module, the service used by each module, the prompting techniques activated in the query generator, and the parameters of each component, is expressed in a configuration file.

The system takes a natural language question as input and produces an answer through five sequential stages.

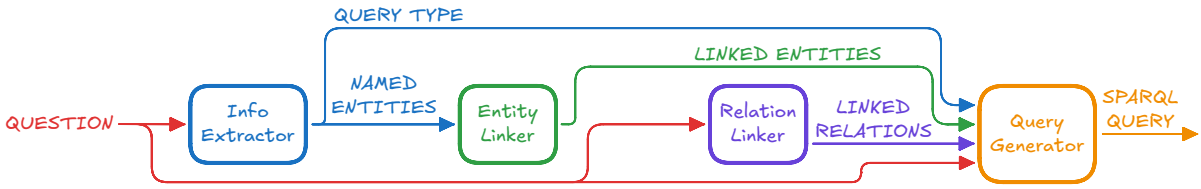

Figure 3.1: *High-level overview of the proposed KGQA pipeline.*

The Information Extractor invokes an LLM with structured output to extract from the question the named entities mentioned in it (persons, publications, and venues) and the type of query that the system is expected to generate, chosen from the set of query types defined by the DBLP-QuAD dataset [3] (`SINGLE_FACT`, `COUNT`, `NEGATION`, …).

The Entity Linker maps each named entity to a URI in the DBLP Knowledge Graph by querying the dblp search API and selecting the best match for each candidate.

For both stages, a gold variant that reads the correct output directly from the dataset is also available, so that the ablation study of chapter 4 can evaluate the downstream modules independently of the quality of the preceding stages.

The Relation Linker selects the ontology predicates that are relevant to the question, and is implemented in two variants: a static one that includes the full DBLP ontology in every prompt, and a dynamic one based on RAG that retrieves only the most relevant predicates for the current question.

The Query Generator builds a prompt that combines a system prompt with the linked entities, the linked predicates, and a configurable set of prompting techniques (static few-shot examples, dynamic few-shot examples retrieved from the training set, query-type filtering, and Chain-of-Thought reasoning), and asks the LLM to produce a SPARQL query under a structured output schema.

Finally, the Query Executor sends the query to a SPARQL endpoint and returns the result to the user.

3.2 Design of the Proposed System

3.2.1 System Architecture

The system is implemented as a Python package (`dblp_kgqa`) organized in two layers. The lower layer is a set of services that wrap access to the infrastructure on which the pipeline depends: a Large Language Model, an embedding model, a Neo4j database, and the DBLP-QuAD dataset. The upper layer is a set of modules that implement the five stages of the pipeline introduced in section 3.1, and that use the services to do their work.

We use three design patterns to organize this architecture. The Strategy pattern is used inside each pipeline module: every module defines an abstract base class with a single callable interface, and admits more than one concrete implementation that the rest of the system treats interchangeably. The Factory pattern is used to build services and modules from a configuration object: a factory class maps the configuration to the corresponding concrete class. Dependency injection is used so that each module receives, at initialization time, the instances of the services it depends on, instead of creating them internally.

3.2.1.1 Services

A service is a class that wraps a piece of infrastructure shared across modules. The four services of the system are the following:

- `LLMService` invokes a Large Language Model and returns its response under a structured-output schema. It is used by the LLM-based Information Extractor presented in section 3.2.2 and by the Query Generator presented in section 3.2.5.
- `EmbeddingService` computes the embedding of a piece of text. It is used by the two retrieval-based components of the system: the RAG relation linker presented in section 3.2.4 and the dynamic few-shot retrieval in the Query Generator presented in section 3.2.5.
- `Neo4jService` provides access to a Neo4j graph database. It hosts the DBLP ontology enriched with the embeddings of the properties descriptions, used by the RAG relation linker of section 3.2.4, and the DBLP-QuAD training set enriched with the embeddings of the natural-language questions, used by the dynamic few-shot retrieval in the Query Generator of section 3.2.5.

- `DblpQuadService` loads the DBLP-QuAD dataset and gives access to its splits. It is used by the gold variants of the upstream modules and by the experiment runner described in chapter 4.

Each service is associated with a configuration class that lists its parameters and is passed to the service constructor at initialization. Every service configuration carries a `type` field with a literal value identifying it (`"LLM"`, `"Embedding"`, `"Neo4j"`, `"DblpQuad"`).
Services are created by the `ServiceFactory` that reads the value of `type` on a configuration and builds the corresponding service class, passing that configuration to its constructor.
All service configurations are listed in `services.yml`, the YAML file that declares which service instances are available to the system.
At startup, the `ServiceRegistry` iterates over these entries and for each one asks the `ServiceFactory` to build the corresponding service, storing the resulting instance under the name declared in the file. The registry holds these instances for the rest of the run and exposes them to the modules: a module configuration declares the names of the services it needs, and the registry retrieves them when the module is built.

With this design, the same service instance is shared by every module that requests it (for example, the same Neo4j connection is used by both the dynamic relation linker and the dynamic few-shot retrieval in the Query Generator), and switching to a different instance of a service (e.g., Gemini LLM or Qwen LLM) is a matter of changing a string in a YAML file.

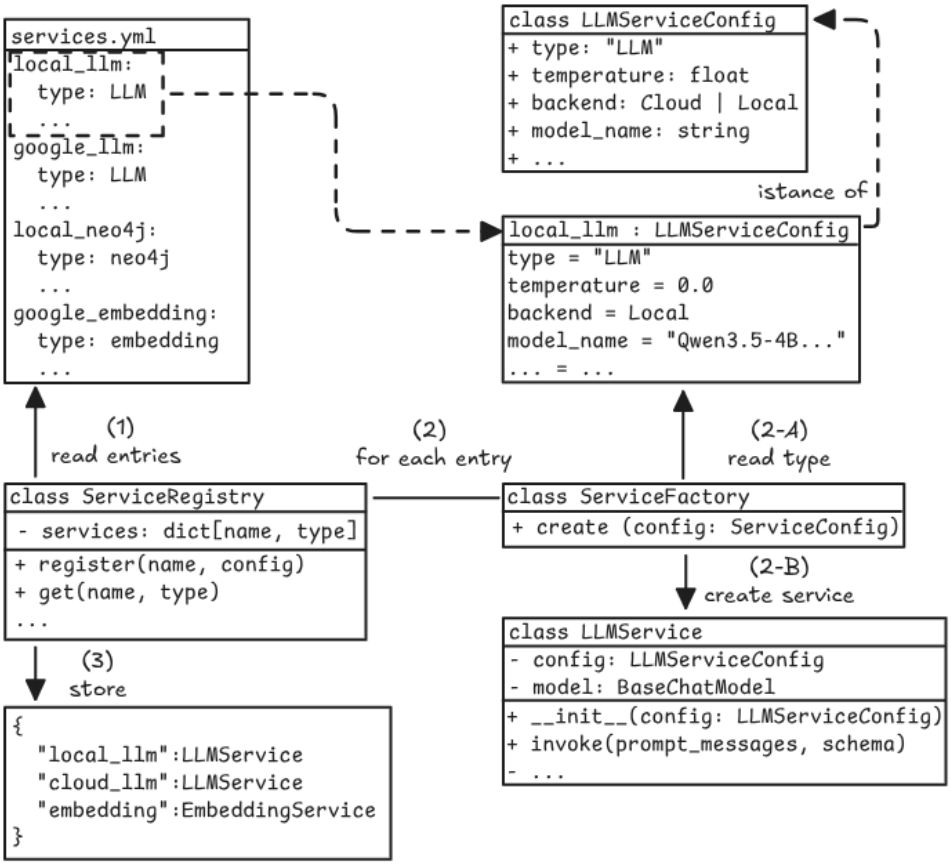

Figure 3.2: *Simplified diagram of the initialization flow of the service layer, illustrated for* `LLMService`. *At startup, the* `ServiceRegistry` *reads each entry of* `services.yml` *(1) and, for each entry, asks the* `ServiceFactory` *to build the corresponding service (2): the factory reads the* `type` *field of the configuration (2-A) and constructs the service class associated with it (2-B). The resulting instance is then stored in the registry under the name declared in the file (3), so that any module that later requests that name retrieves the same instance.*

The figure illustrates this mechanism on `LLMService`, which internally supports two backends, selected via the `backend` field of `LLMServiceConfig`: a Google backend that calls Gemini in the cloud through `langchain-google-genai`, and a LlamaCpp backend that calls a llama.cpp server through its OpenAI-compatible API. Each backend accepts the parameters of its own provider, for example the thinking level for Gemini and the sampling parameters recommended for the Qwen quantization. Both backends are exposed through the same `invoke(messages, schema)` method, so that any module that uses `LLMService` is unaware of which backend is in use.

3.2.1.2 Modules and pipeline

The five pipeline modules implement the stages of the pipeline.

Each module is defined by an abstract base class (`BaseInfoExtractor`, `BaseEntityLinker`, `BaseRelationLinker`, `BaseQueryGenerator`, `BaseQueryExecutor`) that declares a single method `__call__(pipeline_output) → output`, where `output` is the schema fixed for that module (`ExtractedInfo`, `LinkedEntities`, `LinkedRelation`, `GeneratedQuery`, `SparqlResult`). Every concrete implementation of a module must produce an instance of this schema, which acts as the contract between modules.

Each module is associated with a configuration class that lists its parameters and is passed to the module constructor at initialization, alongside the instances of the services that the module depends on.
Every module configuration carries a `strategy` field with a literal value identifying the concrete implementation.
A per-module factory (`InfoExtractorFactory`, `EntityLinkerFactory`, `RelationLinkerFactory`, `QueryGeneratorFactory`, `QueryExecutorFactory`) reads the value of `strategy`, retrieves from the `ServiceRegistry` the services declared by name in the configuration, and constructs the chosen concrete class with both the configuration and the injected services.

The five module strategies that compose a complete pipeline are declared in a pipeline configuration file (`experiment.yml` or `demo.yml`, depending on the use case described in section 3.3.1), one entry per stage. At startup, the `PipelineFactory` reads this file and, for each entry, delegates to the corresponding per-module factory the construction of the chosen strategy; once all five modules are built, it constructs the `Pipeline` by injecting them in its constructor.

The `Pipeline` exposes two methods to invoke the modules on a question: `run` calls the five modules sequentially and returns the final answer, while `run_iter` yields control after each stage so that the demo introduced in section 3.2.7 can let the user review the partial result before passing it to the next module. In both cases, the per-module outputs are collected into a single `PipelineOutput` object that carries the complete trace of the inference and is persisted as the result of a run.

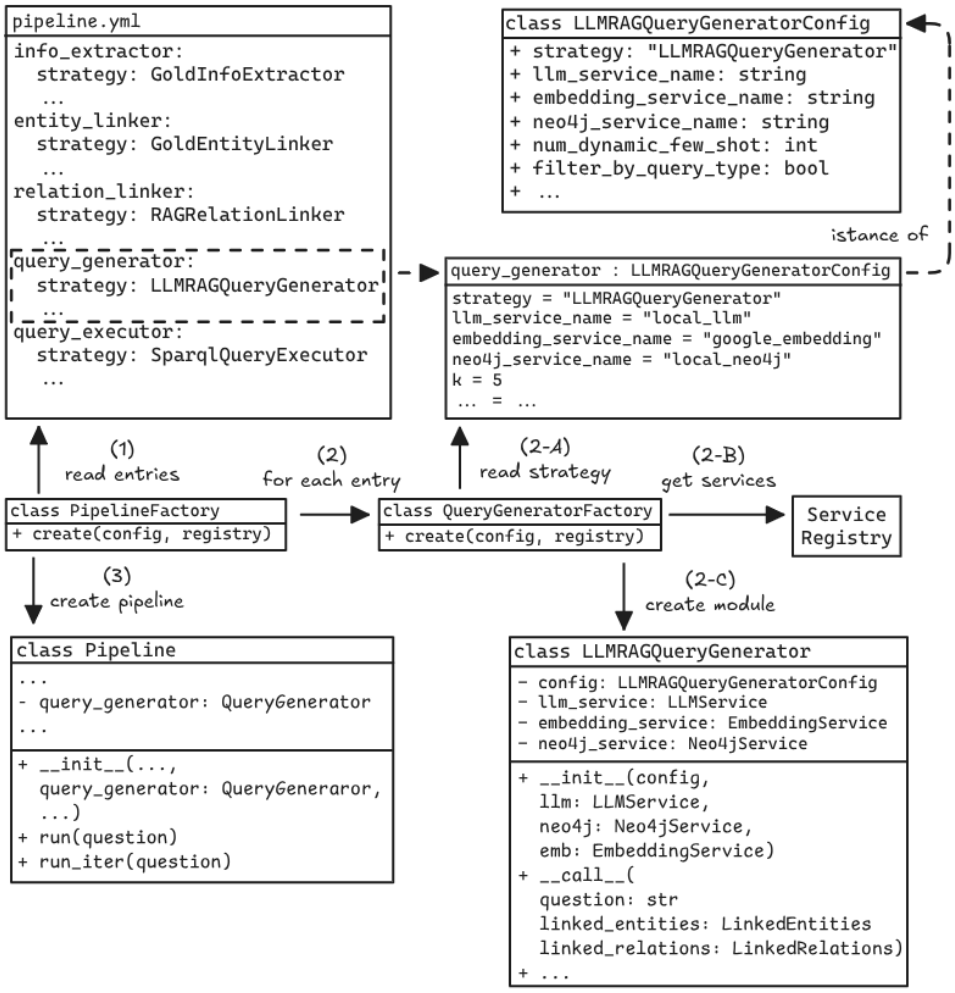

Figure 3.3: *Simplified diagram of the initialization flow of a module and of the pipeline as a whole, illustrated for the Query Generator stage. At startup, the* `PipelineFactory` *reads each entry of the pipeline configuration file (1) and, for each entry, asks the corresponding per-module factory to build the chosen strategy (2): the factory reads the* `strategy` *field of the module configuration (2-A), retrieves from the* `ServiceRegistry` *the services declared by name in the configuration (2-B), and constructs the module by passing both the configuration and the services to its constructor (2-C). Once all five modules are built, the* `PipelineFactory` *constructs the* `Pipeline` *by passing the modules to its constructor (3).*

The figure illustrates this mechanism on the Query Generator, which is implemented in two strategies: `LLMQueryGenerator` and `LLMRAGQueryGenerator`, the second of which extends the prompt with examples retrieved from the training set via Retrieval-Augmented Generation. The two strategies differ in the services they depend on: `LLMQueryGenerator` declares only `llm_service_name`, while `LLMRAGQueryGenerator` declares also `embedding_service_name` and `neo4j_service_name`, since it needs the embedding service and the Neo4j connection to perform the retrieval step.

The next subsections describe each of the five modules in detail.

3.2.2 Information Extractor

The Information Extractor takes the natural-language question as input and produces an `ExtractedInfo` object with two fields: the `query_type` predicted for the question, chosen from the ten types defined by DBLP-QuAD dataset [3], and the list of `named_entities` mentioned in the question, each with a `name` and a `type` in `{person, venue, publication}`.

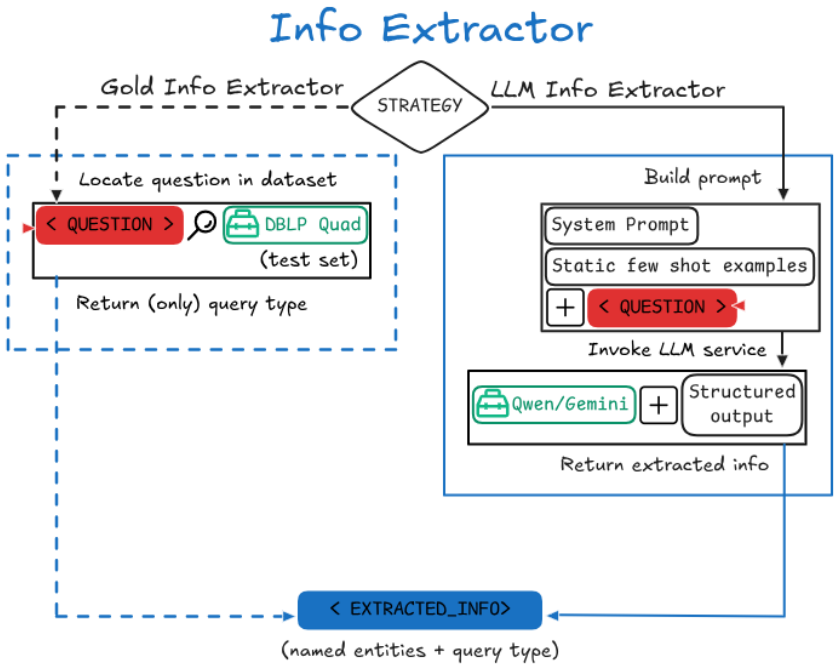

Figure 3.4: *Overview of the Information Extractor module*

`LLMInfoExtractor` and `GoldInfoExtractor` are the two strategies that implement the module. The first invokes a Large Language Model to classify the question and extract the entities, and uses `LLMService`. The second reads the gold query type from the dataset by looking up the question text, and uses `DblpQuadService`. Unlike the Query Generator and the Relation Linker, the Information Extractor is not part of the ablation study of chapter 4: it has been evaluated only qualitatively to support the demo of section 3.2.7, and the comparisons of chapter 4 always rely on the gold variant.

3.2.2.1 Query Types

The ten query types of DBLP-QuAD correspond to ten different structural patterns of the gold SPARQL query. They are listed below.

1. SINGLE_FACT: a question that can be answered using a single fact.
   Example: “What are the papers written by the person Sabrina Senatore?”
2. MULTI_FACT: a question that requires connecting two or more facts to answer.
   Example: “In which venues has Wazir Muhammad published?”
3. DOUBLE_INTENT: a question that poses two user intentions, usually about the same subject.
   Example: “Mention the papers published by Wazir Muhammad and in which year.”
4. BOOLEAN: a question on whether a given fact is true or false.
   Example: “Did Ming-Wang Cheng publish the paper ‘Individual Cell Equalization for Series Connected Lithium-Ion Batteries’?”
5. NEGATION: a question that negates the answer to a Boolean question.
   Example: “Was the paper ‘A Priority Queue Transform’ not published by the person Michael L. Fredman?”
6. DOUBLE_NEGATION: a question that negates the answer to a Boolean question twice.
   Example: “Wasn’t the paper ‘Modeling Syntactic Complexity with P Systems: A Preview’ not not published by the person named Benedek Nagy?”
7. UNION: a question that covers a single intent for multiple subjects at the same time.
   Example: “In VL/HCC and Comput. Music. J., what papers did S. Conversy publish?”
8. COUNT: a question about the count of occurrences of facts.
   Example: “Report the count of papers that Fanggang Wang has published in IEEE Wirel. Commun..”
9. SUPERLATIVE+COMPARATIVE: a question that asks for the maximum or minimum for a subject (superlative) or compares values between two subjects (comparative).
   Example: “When was the first paper by Tom Tollenaere published?”
10. DISAMBIGUATION: a question that requires identifying the correct subject in the question, often by relying on a partial description such as a keyword extracted from the title of a publication, instead of the full title.
    Example: “Mention the publication on neural networks that Sabrina Senatore published in ECAI.”

3.2.2.2 Named Entities

A named entity is a span of the question that refers to a specific resource of the DBLP knowledge graph, and that the Entity Linker maps to a URI in the next stage. The Information Extractor recognizes three categories: persons (authors and editors), venues (journals and conferences), and publications (papers and books). The category is attached to each entity as the `type` field and routes the Entity Linker to the corresponding DBLP search endpoint, as described in section 3.2.3.

3.2.2.3 LLM Information Extractor

`LLMInfoExtractor` builds a prompt composed of a system prompt and a fixed set of few-shot examples, and asks the LLM to fill a structured output schema.

The system prompt encodes the entity categories of section 3.2.2.2 with two exclusions:

- Institutions and universities, such as MIT or ETH Zürich, are sometimes mentioned in the question but are not modeled as venues in DBLP, and are not extracted.
- Descriptive phrases about a topic, such as "neural networks" in a question of the DISAMBIGUATION type, are not publications and are not extracted either, because the publication is the unknown of the query rather than a named entity to be linked.

The system prompt also lists the language signals that distinguish each query type, such as "how many" for COUNT, "first" or "most" for SUPERLATIVE+COMPARATIVE, "not not" for DOUBLE_NEGATION, and a descriptive topic phrase for DISAMBIGUATION.

The few-shot examples are ten, one for each query type, and follow the format reported in section 3.2.2.1.

The structured output schema includes a free-text `explanation` field placed before the two output fields, so that the LLM produces a Chain-of-Thought reasoning that identifies the entities and motivates the chosen query type before committing to the final classification, following the same pattern adopted by the Query Generator in section 3.2.5.3.

3.2.2.4 Gold Information Extractor

`GoldInfoExtractor` is the variant used in the ablation study. At initialization, it loads the chosen split of DBLP-QuAD through `DblpQuadService` and builds an in-memory index from the question text to the gold query type. At inference time, it looks up the user question in this index and returns the gold query type.

3.2.3 Entity Linker

The Entity Linker takes the named entities produced by the Information Extractor and returns a `LinkedEntities` object: a list of `LinkedEntity` items, each with the entity name, the URI assigned to it in the DBLP knowledge graph, and the entity type.

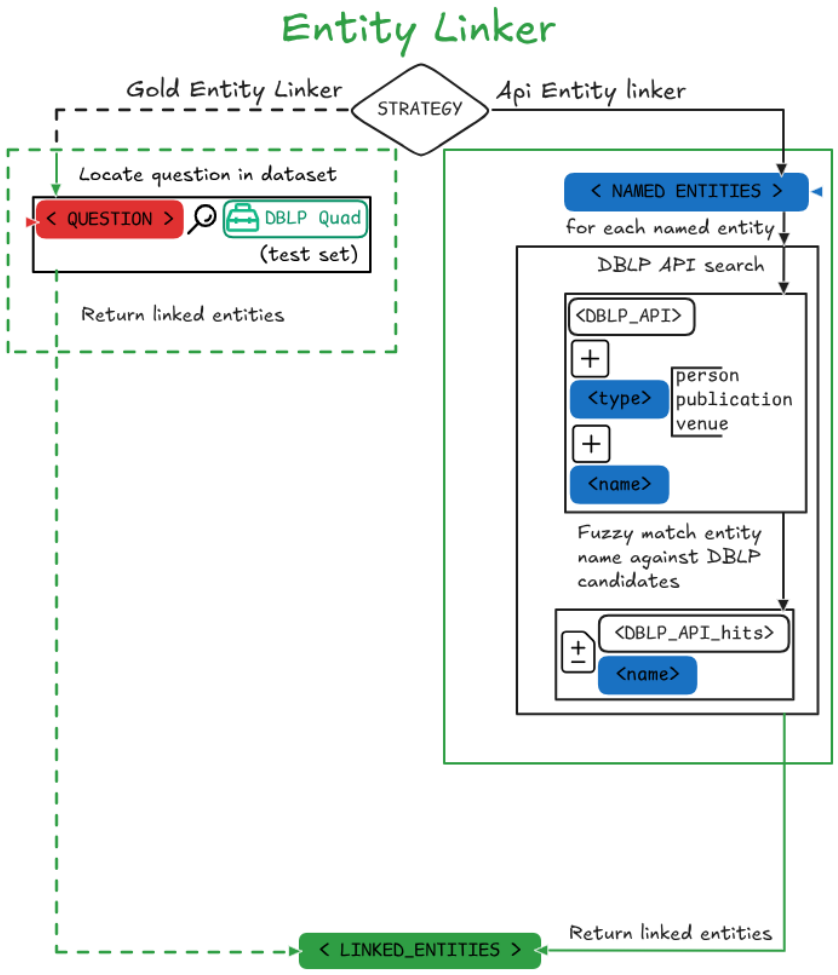

Figure 3.5: *Overview of the Entity Linker module*

`ApiEntityLinker` and `GoldEntityLinker` are the two strategies that implement the module. The first calls the dblp search API to find the URI of each named entity, and uses no service. The second reads the gold URIs from the DBLP-QuAD dataset, and uses `DblpQuadService`. Like the Information Extractor, the Entity Linker is not part of the ablation study of chapter 4: `ApiEntityLinker` has been evaluated only qualitatively to support the demo of section 3.2.7, and the comparisons of chapter 4 always rely on the gold variant.

3.2.3.1 API Entity Linker

The dblp search API exposes three endpoints, one for each named entity type: `https://dblp.org/search/author/api` for persons, `https://dblp.org/search/publ/api` for publications, and `https://dblp.org/search/venue/api` for venues. For every named entity in input, `ApiEntityLinker` sends a request to the endpoint that matches the entity type, passing the entity name and asking for a JSON response.

The response contains a list of candidate hits. Each hit carries a textual label, namely the author name, the publication title, or the venue name depending on the endpoint, and the URI of the candidate in dblp. To select the best match, the strategy applies the `difflib.get_close_matches` function to the input entity name and the labels of the candidates. The function returns the candidate label most similar to the input name, provided that the similarity reaches a fixed cutoff of 0.3, and the URI of the selected candidate is returned. The cutoff value is taken from NLQxform [27], which also queries the dblp search API with a similar mechanism.

3.2.3.2 Gold Entity Linker

`GoldEntityLinker` is the variant used in the ablation study. At initialization, it loads the chosen split of DBLP-QuAD through `DblpQuadService` and builds an in-memory index from the question text to the gold entities. At inference time, it looks up the user question in this index and returns each gold entity with its type inferred from the URI prefix: `/pid/` for persons, `/rec/` for publications, otherwise a venue.

3.2.4 Relation Linker

The Relation Linker selects the DBLP ontology predicates that the Query Generator is restricted to use.

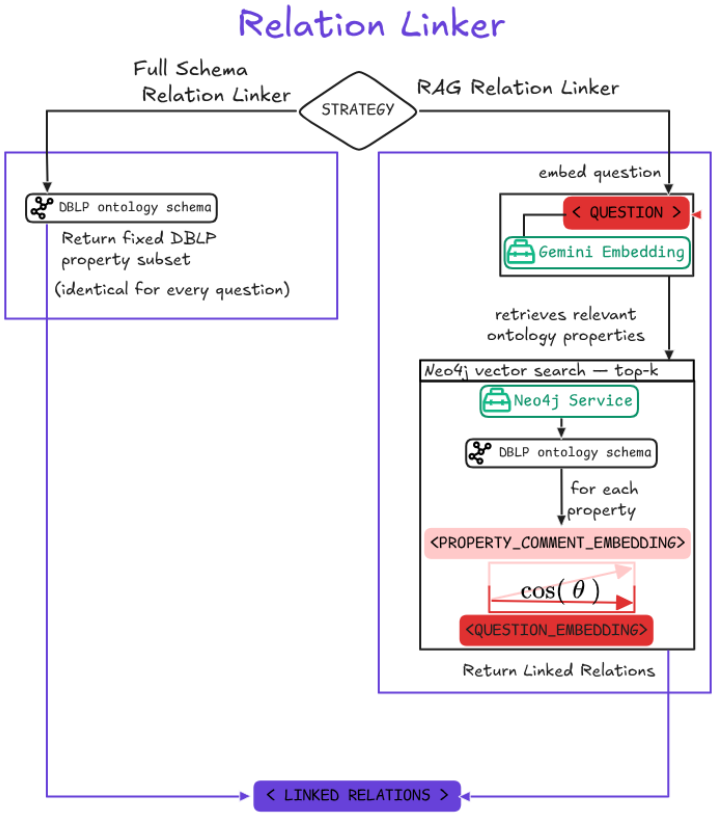

Figure 3.6: *Overview of the Relation Linker module*

`FullSchemaRelationLinker` and `RAGRelationLinker` are the two strategies that implement the module. The first is the baseline: it returns the entire DBLP ontology every time it is called, leaving the Query Generator to pick the relevant predicates from the full schema. The strategy uses no service.

`RAGRelationLinker`, instead, runs a vector similarity search before returning the output: only the predicates most relevant to the user question are retrieved and passed to the Query Generator. To perform this Retrieval-Augmented Generation step, the strategy uses two services: `EmbeddingService`, which embeds the user question, and `Neo4jService`, which hosts the DBLP ontology with the pre-computed embeddings of the predicates.

3.2.4.1 Full-Schema Relation Linker

The listing returned by `FullSchemaRelationLinker` pairs each predicate with its IRI and the natural-language description taken from the official ontology. It is sourced from the one used by ASK-DBLP, the most recent system analyzed in chapter 2, and covers almost the entire DBLP ontology, with three exclusions: `authorOf`, `creatorOf`, and `editorOf`. These are the active forms of the inverse predicates `authoredBy`, `createdBy`, and `editedBy`, which are the only forms used by the DBLP-QuAD gold queries. Removing them prevents the LLM from picking a form that does not match the gold queries. The same exclusion is applied to the predicates handled by the RAG strategy.

3.2.4.2 RAG Relation Linker

The implementation of `RAGRelationLinker` is split into two phases: an offline preprocessing pipeline that prepares the ontology for retrieval, and an online step that runs for every user question.

The offline phase begins with the import of the ontology into Neo4j through the n10s plugin, which converts the schema into a graph. Each property declared in the TTL file of the schema becomes a node labeled `Relationship`, with the URI, the local name, and the natural-language description of the property as node attributes; each entity class becomes a node labeled `Class`; the `rdfs:domain` and `rdfs:range` declarations become `DOMAIN` and `RANGE` relationships connecting each property node to the class that can appear as subject and as object of a triple using that property.

Once the ontology is in Neo4j, the preprocessing pipeline queries the graph for the `Relationship` nodes that carry a description, builds a text for each one in the form `name: description`, and embeds the texts with `gemini-embedding-001`. The same three active forms excluded from the listing of section 3.2.4.1 are excluded here too. Each resulting embedding is attached to the corresponding `Relationship` node as a vector property, so that all the data needed by the online retrieval (the property name, its IRI, its description, its domain and range, and its embedding) live on the same node.

The `gemini-embedding-001` model accepts a `task_type` parameter[1] that controls how the embedding is computed: different values produce different embeddings of the same text, each one optimized for a specific retrieval scenario. Two of these values come as a pair: `RETRIEVAL_DOCUMENT` is meant for the items that populate a corpus to be searched, while `RETRIEVAL_QUERY` is meant for the queries used to search the corpus. The pair is designed for cases in which the two sides of the comparison are of different nature, such as a question on one side and a property description on the other. Both task types place their output in a shared space where cosine similarity measures how well a document matches a query, despite the two embeddings being computed differently. The offline embedding of each property uses `RETRIEVAL_DOCUMENT`, since the properties form the corpus that the user question will search.

The online phase, performed for every user question, follows the three steps described below.

[1] https://cloud.google.com/vertex-ai/generative-ai/docs/embeddings/task-types

The first step embeds the user question with `gemini-embedding-001` using task type `RETRIEVAL_QUERY`.

The second step computes the cosine similarity between the question embedding and the embedding of every `Relationship` node, sorts the nodes by score in decreasing order, and keeps the first k.

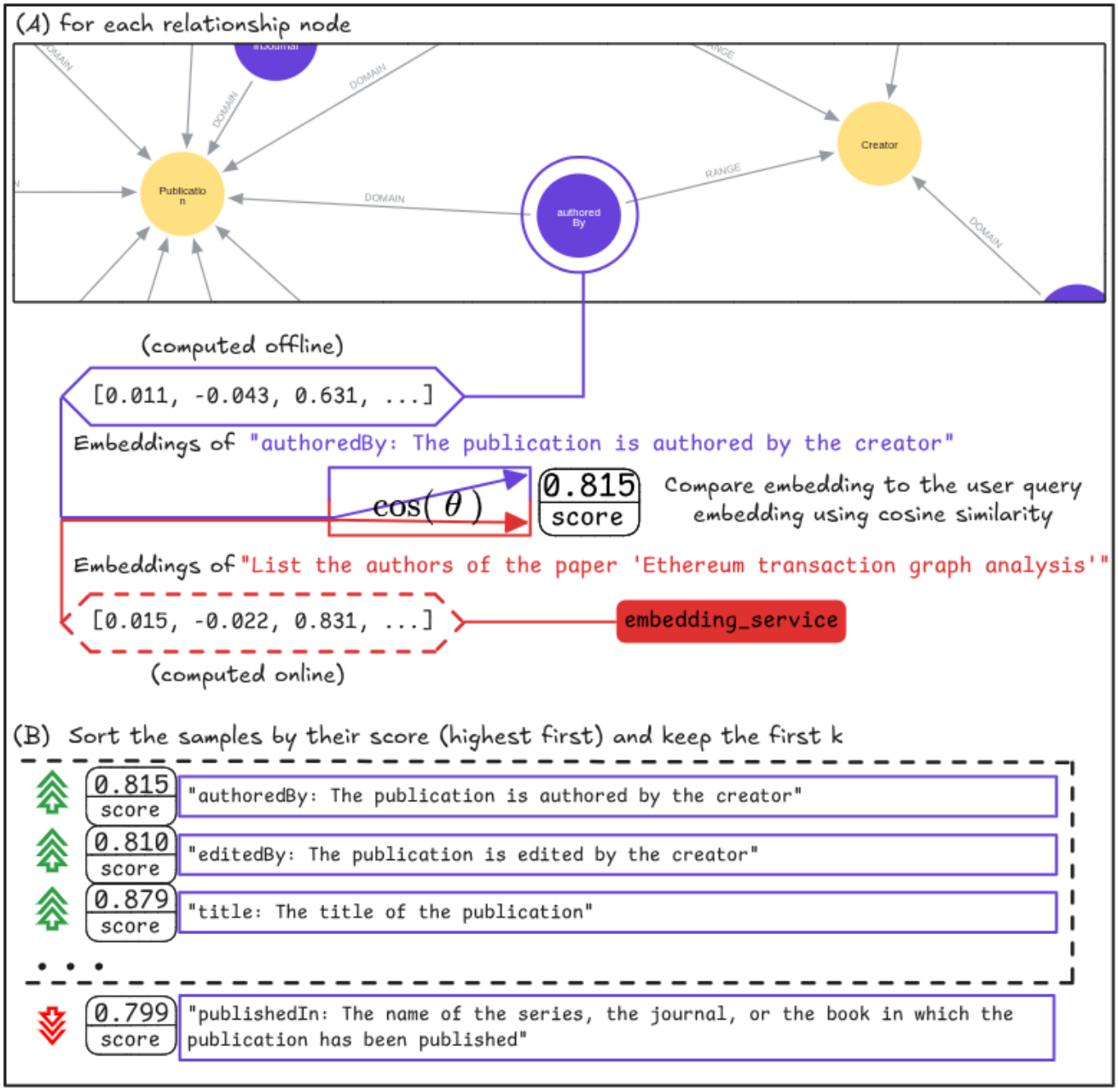

Figure 3.7: *Cosine similarity is computed between the user question embedding and the embedding of every* `Relationship` *node; the nodes are sorted by score in decreasing order, and the first k are kept while the rest are discarded.*

The third step is the optional enrichment of each retained property with its domain and range. When the enrichment is enabled, each property is returned with its IRI, its description, its domain, and its range; when it is disabled, only the IRI and the description are returned.

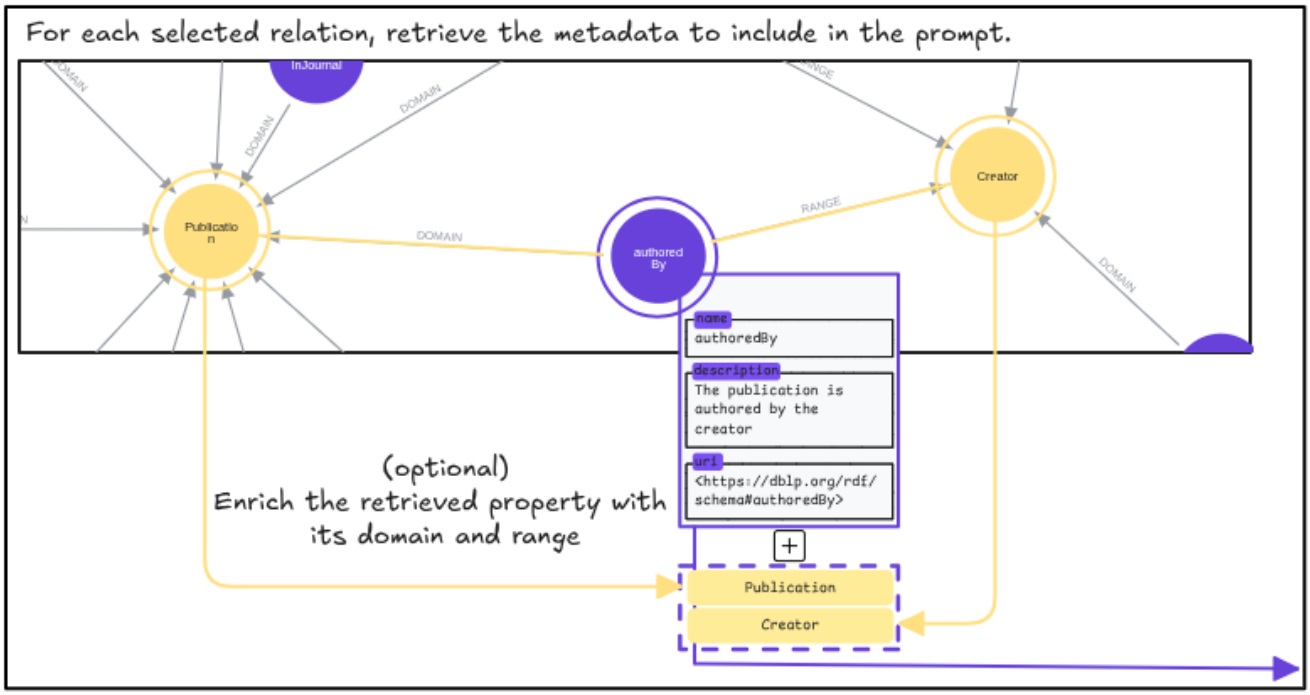

Figure 3.8: *For each retained property, when the enrichment is enabled, the connected* `Class` *nodes are collected through the* `DOMAIN` *and* `RANGE` *relationships and returned alongside the IRI and the description.*

The retrieved properties are passed to the Query Generator.

3.2.5 Query Generator

The Query Generator takes the question, the linked entities, and the linked relations as input. Its output is a `GeneratedQuery` instance that contains the SPARQL query.

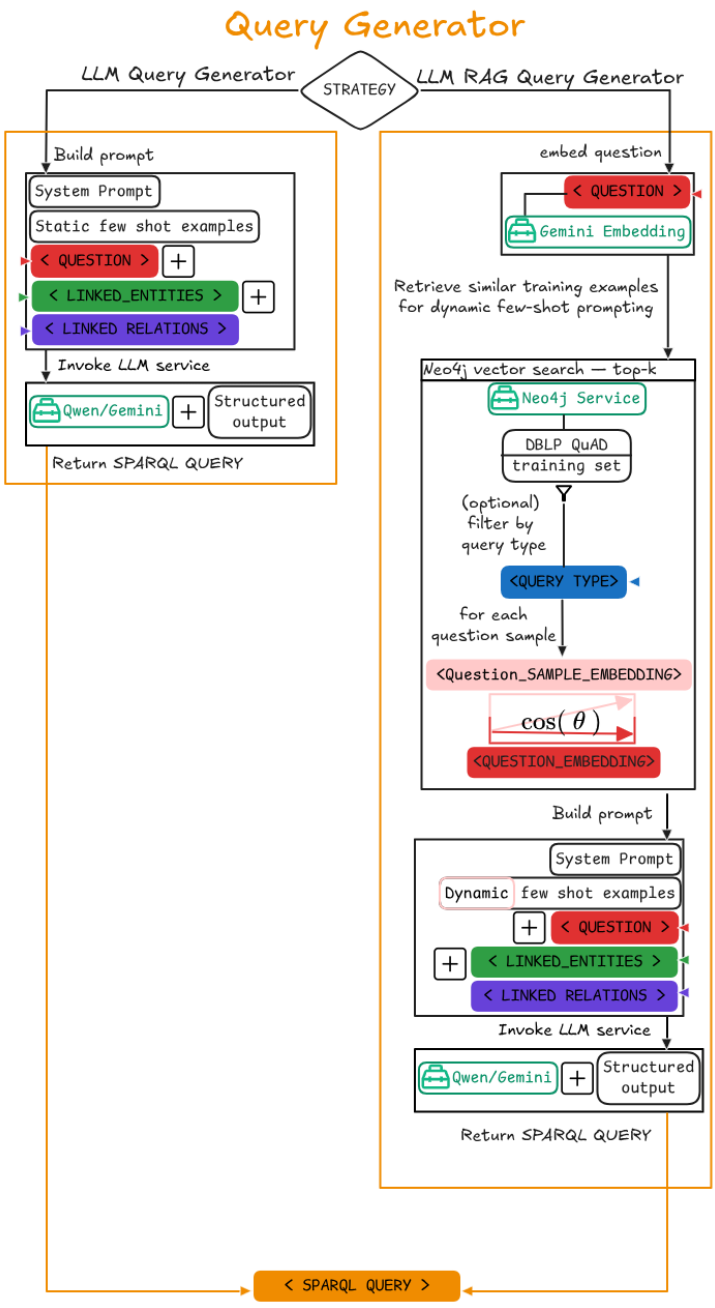

Figure 3.9: *Overview of the Query Generator module.*

`LLMQueryGenerator` and `LLMRAGQueryGenerator` are the two strategies that implement the module. The first is the baseline. The prompt it sends to the LLM is composed of a system prompt, optionally followed by a set of hand-crafted few-shot examples, and ends with the user question together with the linked entities and relations. The LLM output follows a structured output schema

that contains the SPARQL query and, when configured to do so, a step-by-step explanation that the LLM fills before the query, prompting it to apply Chain-of-Thought reasoning. To do its work, the strategy uses `LLMService`.

`LLMRAGQueryGenerator` extends the baseline with one further step: before invoking the LLM, a vector similarity search retrieves additional few-shot examples from the DBLP-QuAD training set and appends them to the prompt, optionally restricted to the examples whose query type matches the one predicted by the Information Extractor. To perform this Retrieval-Augmented Generation step, the strategy uses two further services: `EmbeddingService`, which embeds the user question, and `Neo4jService`, which hosts the training set with the pre-computed question embeddings.

3.2.5.1 System Prompt

The system prompt is the first part of the prompt sent to the LLM. It serves two purposes: telling the LLM how to read the linked entities and the linked relations that will appear in the user message, and stating the rules that the generated SPARQL query must follow.

The first part of the system prompt is a textual description of the format in which the linked entities and the linked relations will be provided in the user message. The description is not fixed: it depends on the entity linker and the relation linker strategies in use, since the format and the meaning of their outputs change with the strategy. For example, when the relation linker is the full-schema one, the description tells the LLM that the relations field contains the entire DBLP ontology and that the predicates to use must be picked from this complete list; when the relation linker is the RAG-based one, the description tells the LLM that the relations field contains only a subset of predicates retrieved by semantic similarity to the question, and that the LLM is restricted to choosing among them, and informs the LLM that each predicate is accompanied by its domain and range, which constrain the entity types that can appear as subject and object in the corresponding triple.

The rest of the system prompt is a list of eight rules. Although the few-shot examples described in the next subsection already illustrate most of the conventions to follow, explicit rules are useful for two reasons. First, they encode the conventions of the DBLP-QuAD dataset that cannot be inferred from general SPARQL knowledge, such as the use of the passive form of the DBLP predicates in the gold queries (a publication is the subject of `authoredBy`, not its object). Second,

they cover situations that may not appear in every example, such as negation, temporal filters, or unions. The rules were formulated by inspecting the DBLP ontology, the DBLP-QuAD gold queries on a subset of the validation set, and the recurrent issues that the LLM was most likely to miss.

The rules are described below together with their motivation (the literal text used in the prompt is available in the public repository of the project).

1. Target variables

   The query must use a fixed set of variable names in the SELECT clause: `?answer` for single-answer fact retrieval, `?firstanswer` and `?secondanswer` for questions that ask for two related pieces of information, and `(COUNT(DISTINCT ?answer) AS ?count)` for counting. Descriptive names such as `?year`, `?venue`, or `?paper` are not allowed. This convention is required to be compatible with the DBLP-QuAD F1 metric defined by Banerjee et al., one of the metrics adopted to compare the system against the baselines of chapter 4, which reads the answer from the SPARQL result by looking for these specific variable names; if the LLM uses different names, the answer cannot be extracted and the query is counted as wrong even when the result is correct.

2. Predicate direction

   The DBLP ontology defines for most relations both an active and a passive form, for example `authorOf` (from creator to publication) and its inverse `authoredBy` (from publication to creator). The DBLP-QuAD gold queries adopt the passive form throughout, so that the subject of a triple is the publication for predicates that describe the publication and the person for predicates that describe the person. For example, the relation between a publication and one of its authors is always written `<publication> <authoredBy> <person>`, never the other way around. The rule states this general convention and gives examples for the predicates most likely to confuse the LLM, to prevent it from picking the active form when the gold query uses the passive one.

3. URIs

   URIs must appear as full IRIs in angle brackets, for example `<https://dblp.org/rdf/schema#authoredBy>`. Namespace prefixes such as `dblp:authoredBy` are not allowed, since the system does not declare any prefix in the SPARQL queries it sends to the endpoint and the few-shot examples are formatted with full IRIs. The rule also requires the query to use only

the URIs that appear in the linked entities and the linked relations of the current question, to prevent the LLM from inventing identifiers that may not exist in the knowledge graph.

4. Venues

   A venue can be represented in two forms, depending on what the entity linker provides. When the linked entity carries a URI, the predicate `publishedInStream` is used with the URI in angle brackets, as in `?x <https://dblp.org/rdf/schema#publishedInStream> <https://dblp.org/streams/conf/ecai>`. When the linked entity carries only a name and no URI, the predicate `publishedIn` is used with the name as a literal string, as in `?x <https://dblp.org/rdf/schema#publishedIn> 'ECAI'`. The choice between the two forms is therefore determined by the form of the linked entity.

5. Negation in boolean questions

   The DBLP-QuAD dataset adopts a specific convention for boolean questions involving negation, with two forms depending on the number of negations in the question. For a single negation, the triples that represent the fact appear both in the body of the ASK query and inside a `FILTER NOT EXISTS` clause that repeats them, as in `ASK { <pub> <authoredBy> <person> FILTER NOT EXISTS { <pub> <authoredBy> <person> } }`. For a double negation, where the two negations cancel each other out, the query reduces to a plain `ASK { ... }` on the same triples without any FILTER clause. The rule lists both forms so that the LLM can match the appropriate one to the question.

6. Temporal filters

   Questions that refer to a relative time frame, such as “in the last N years”, must use the SPARQL function `YEAR(NOW())` rather than hardcoded years, as in `FILTER(xsd:integer(?year) ≥ YEAR(NOW()) - N)`. This keeps the queries valid as time passes and avoids the LLM committing to a specific year drawn from its training data.

7. UNION

   Each branch of a UNION must be wrapped in its own pair of curly braces, as in `{ branch1 } UNION { branch2 }`, and the constraints that apply to both branches, such as a shared author filter, must be placed outside the UNION rather than repeated inside each branch. The rule keeps each branch syntactically isolated and factors out the constraints that are shared between them.

8. Query format

   SPARQL queries with a non-boolean answer must use `SELECT DISTINCT ... WHERE { ... }`, while boolean questions must use `ASK { ... }`. Two mixed forms are explicitly forbidden: `ASK { SELECT ... }`, an unusual construction excluded to keep the output format predictable, and `ASK { ... MINUS ... }`, which is an alternative way to express negation that would compete with the pattern required by rule 5.

3.2.5.2 Static Few-Shot Examples

The static few-shot set consists of one example for each query type defined by DBLP-QuAD and described in section 3.2.2, so that the LLM sees at least one demonstration of every structural pattern it may be asked to produce. Each example is drawn from the training set of DBLP-QuAD.

Each example, in the format that the user message takes at runtime, includes the question, the entities and relations linked from the question, the SPARQL query, and optionally a step-by-step reasoning (Chain-of-Thought), described in the next subsection. The examples are inserted in the prompt as a sequence of alternating human and ai messages placed between the system prompt and the current question, so that the LLM perceives them as previous turns of a conversation rather than as material accumulated in a single block. The following figure shows an example.

3.2.5.3 Chain-of-Thought

When the Chain-of-Thought technique is active, the structured output schema sent to the LLM includes a field for the step-by-step reasoning that produces the SPARQL query. The LLM is asked to fill this field before the query, so that it formulates the chain of decisions that lead to the query (the query type, the target variable, the triple patterns and their direction, the special constructs needed) rather than producing the query as a single output. The same field is included in the few-shot examples shown in the prompt, so that the LLM observes the expected form of the reasoning trace and can imitate it.

Chain-of-Thought is a prompting technique that has been shown to improve the quality of the answers produced by LLMs on multi-step tasks [24]. Generating a SPARQL query against the DBLP knowledge graph is multi-step by nature: the LLM must identify the structural pattern of

Figure 3.10: *Static few-shot example for the MULTI_FACT query type.*

the question, choose the correct predicates, decide their direction in each triple, and assemble any special constructs required. The technique is particularly relevant for the consumer-hardware target of the system, the quantized 4B-parameter Qwen 3.5 model: smaller models tend to produce a query that matches the form of the examples without working out the logic that connects the question to the query, and an explicit reasoning trace forces this logic into the output before the query is written.

Some LLMs, including Qwen 3.5, support a native reasoning mode that emits dedicated reasoning tokens before the final answer. This mode is disabled in the configuration of the system. Disabling it is the configuration recommended for the smaller Qwen variants by Unsloth[2], the team that distributes the quantized weights of the model, and preliminary tests confirmed that when enabled the model frequently saturated its output budget with reasoning tokens and failed to produce a valid response. The reasoning role is therefore delegated entirely to the explicit step-by-step field described above, which integrates naturally with the structured output schema. The actual contribution of Chain-of-Thought to the quality of the generated queries on this system is

[2]https://unsloth.ai/docs/models/qwen3.5

quantified by the ablation study presented in chapter 4.

Activating Chain-of-Thought on the few-shot examples requires that every example carries its own reasoning trace. The static examples described in section 3.2.5.2 are accompanied by a hand-written reasoning, but the dynamic examples retrieved from the DBLP-QuAD training set are not: the training set contains 7000 question-query pairs but no reasoning trace. Two alternatives were considered and discarded. Leaving the reasoning field empty in the dynamic examples would create a mismatch between the format of the examples and the format requested in the structured output schema, increasing the risk of malformed outputs. Filling it with a programmatic placeholder (for example, a mechanical paraphrase of the query) would lead the LLM to imitate a superficial pattern rather than to produce a genuine reasoning trace. The chosen solution is therefore to generate a real reasoning trace for every question of the training set offline, before any inference takes place.

The generation follows a teacher-student pattern: a more capable model is used offline to produce a reasoning trace for each of the 7000 questions of the training set, conditioned on the question, the gold SPARQL query, the entities and relations involved, and a small set of hand-crafted examples for the relevant query type. The model used as teacher is Gemini 3 Flash (with the thinking level set to high), the most capable model available to the project, which on standard reasoning benchmarks performs comparably or better than the previous flagship Gemini 2.5 Pro. The teacher is asked to produce a concise, numbered reasoning that follows a fixed format: the first step states the query type and the target variable, the following steps describe each triple of the query in an edge-centric notation of the form `subject --predicate--> object`, and the last step justifies any special construct (FILTER NOT EXISTS, UNION, COUNT, AVG, MIN, BIND/IF) when present. This fixed format keeps each explanation short enough to fit comfortably in the prompt context at inference time, and predictable enough that the small model can imitate the pattern across examples rather than having to interpret traces of varying form. The output is stored alongside the corresponding training-set question in a JSON file, and is then loaded into the Neo4j database that supports the dynamic few-shot retrieval described in section 3.2.5.5.

3.2.5.4 Structured Output

The Query Generator asks the LLM to produce its output as an instance of a fixed schema, instead of a free-form string. The schema is defined as a Pydantic[3] class with named fields, types, and a short description for each field that tells the LLM what to write in it. Both LLMs used in this work, Gemini in the cloud and Qwen running locally through llama.cpp, support this functionality natively: the inference engine constrains the generation so that the output conforms to the schema. The constraint guarantees the shape of the output but not its content (the LLM can still produce a query that is well-formed JSON but semantically wrong).

The Query Generator defines two such schemas, one for each value of the Chain-of-Thought flag. When Chain-of-Thought is disabled, the schema has a single field, the SPARQL query string. When Chain-of-Thought is enabled, the schema has two fields, the step-by-step reasoning and the SPARQL query, in this order, so that the LLM is forced to write the reasoning before the query rather than after it.

The benefits of this technique are twofold. First, the output of the LLM does not need to be parsed with regular expressions or fragile string manipulation: it can be read directly as the corresponding object, with the SPARQL query immediately accessible as a field. Second, the reasoning and the query are kept in two separate fields rather than mixed in a single body of text, which removes the need to split them later and prevents the reasoning from leaking into the SPARQL string sent to the endpoint.

3.2.5.5 Dynamic Few-Shot via Retrieval-Augmented Generation

The Dynamic Few-Shot technique adds to the prompt a set of examples that are not fixed but selected at runtime by semantic similarity to the user question, drawn from the same training set of DBLP-QuAD that contributes one sample per query type to the static few-shot of section 3.2.5.2. Each retrieved example is shown together with its gold SPARQL query and (optionally) its explanation. Compared to a fixed set, the dynamic set surfaces examples that can share with the user question more than the structural pattern: similar entity types, similar phrasings, similar special constructs in the gold query. This is the technique that gives `LLMRAGQueryGenerator` its name, since it follows the Retrieval-Augmented Generation paradigm introduced in chapter 2.

[3] https://github.com/pydantic/pydantic

The technique requires an offline preprocessing pipeline that prepares the training set for retrieval. The DBLP-QuAD training set contains 7000 question-query pairs, and each pair is provided in two natural-language forms, an original question and a semantically equivalent paraphrase. For every form of every question, an embedding is pre-computed once with the same `gemini-embedding-001` model used by the rest of the system. The embedding task type is set to `SEMANTIC_SIMILARITY` because both sides of the comparison performed online are natural-language questions (the user question and a training-set question), and this task type optimizes the embedding for symmetric similarity between two pieces of text of the same nature, rather than for the asymmetric question-to-document setting assumed by the retrieval task types. The embeddings, together with the rest of the training-set data, are then loaded into Neo4j as the index over which the online retrieval is performed.

Each training-set sample is represented in Neo4j as a `QuestionSample` node holding the gold SPARQL query, the linked entities, the linked relations, and the synthetic explanation. The sample is connected to a `QueryType` node by a `HAS_TYPE` relationship, so that the query type predicted by the Information Extractor can be used to filter the retrieval. Two `Question` nodes are attached to the sample, one for the original form and one for the paraphrase, connected by typed relationships (`IS_ORIGINAL_OF` and `IS_PARAPHRASED_OF`); each `Question` node carries the natural-language text of that form together with its embedding as a vector property. Keeping the two forms on two separate nodes, each with its own embedding, lets the retrieval choose at query time which of the two forms is closer to the user question, instead of committing to that choice at preprocessing time, for example by averaging the two embeddings or by keeping only one of them.

The retrieval, performed online for every user question, follows the five steps described below.

The first step embeds the user question. The same `gemini-embedding-001` model used in the offline preprocessing is invoked online with task type `SEMANTIC_SIMILARITY` on the natural-language question, producing a vector that lives in the same space as the embeddings stored in Neo4j and that can therefore be compared to them with cosine similarity.

The second step is the optional Filter by Query Type. When the filter is enabled, the search is restricted to the `QuestionSample` nodes whose `QueryType` matches the query type predicted by the Information Extractor for the user question; when the filter is disabled, all 7000 samples of the training set are considered as candidates. The filter is motivated by the observation that the structural type of the SPARQL query is what determines the demonstration value of an example: a SINGLE_FACT example does not show how to assemble a UNION query, no matter how close the two questions may be in surface form. Although the embedding similarity already tends to rank questions of the same query type higher than questions of a different type, this is not guaranteed, especially when two questions of different types share many surface features (for example “How many papers…” and “What are the papers…” may rank close to each other even though one is COUNT and the other is SINGLE_FACT). The filter ensures that this case cannot happen and acts as a robustness mechanism on top of the similarity ranking.

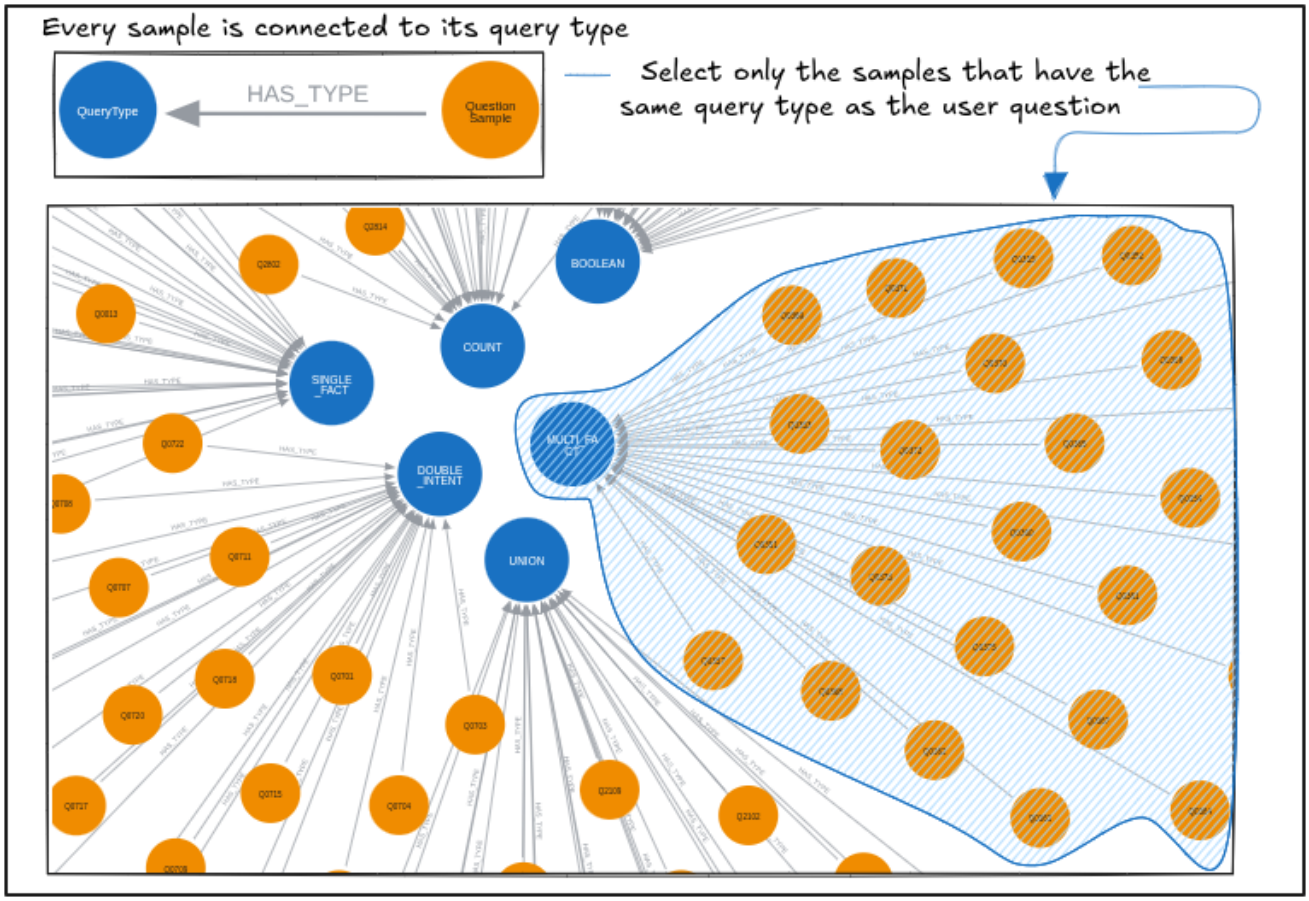

Figure 3.11: *Filter by Query Type: when enabled, only the samples whose query type matches the one predicted by the Information Extractor are kept as candidates.*

The third step computes the cosine similarity between the embedding of the user question and the embedding of every `Question` node in the candidate pool. Since each sample carries two `Question` nodes (original and paraphrase), each sample produces two similarity scores, and a single sample could in principle be matched twice if both its forms ranked high. To avoid this, the retrieval collapses the two scores per sample into one, keeping only the higher of the two as the score of the sample together with the form of the question that achieved it. The example shown in the figure illustrates a case in which the original question wins by a small margin (0.946) over the paraphrase (0.945); in either case, the form selected here is the one that will be shown in the prompt as part of the retrieved example, and the sample appears at most once in the final ranking.

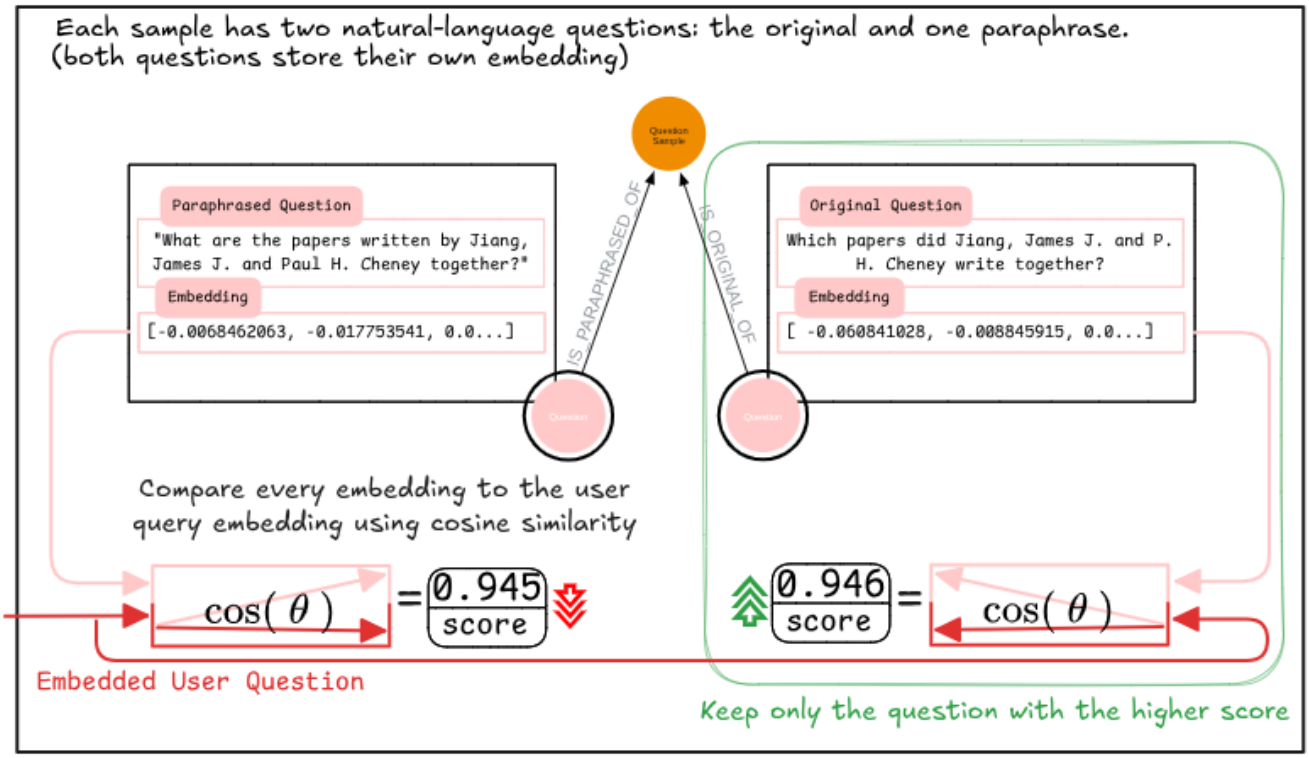

Figure 3.12: *Cosine similarity is computed between the user question embedding and the embeddings of the original and the paraphrased forms of every candidate sample; the higher of the two scores is kept as the score of the sample.*

The fourth step sorts the samples by their score in decreasing order and keeps the first k, where k is a configurable parameter. In most of the experiments of chapter 4 the value used is k = 5; additional runs with k = 3, k = 10, and k = 15 are included in the ablation to study how the size of the dynamic context affects the quality of the generated query.

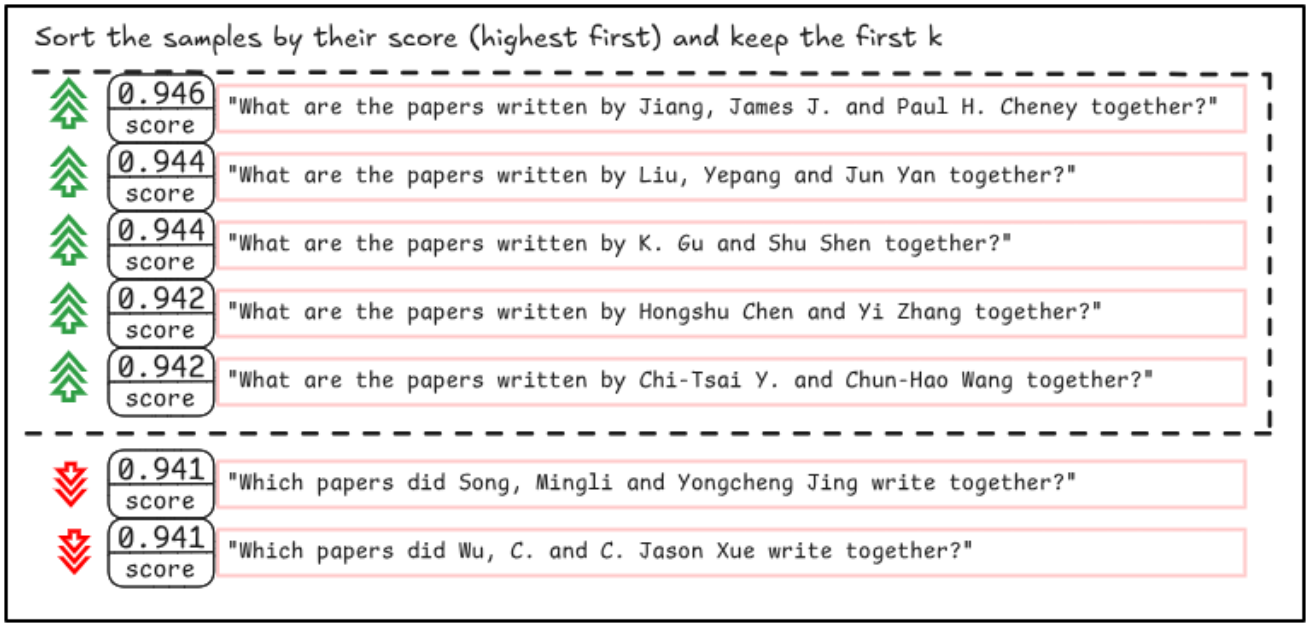

Figure 3.13: *The candidate samples are sorted by score in decreasing order, and the first k are kept while the rest are discarded.*

The fifth step collects, for every retained sample, the metadata that will be included in the prompt: the question text in the form selected at the third step, the linked entities and the linked relations as recorded in the dataset, the gold SPARQL query, and the explanation generated by the pipeline of section 3.2.5.3. These are the same fields that appear in a static few-shot example, so that the dynamic and the static examples are visually indistinguishable to the LLM and can be mixed in the same prompt without format inconsistencies.

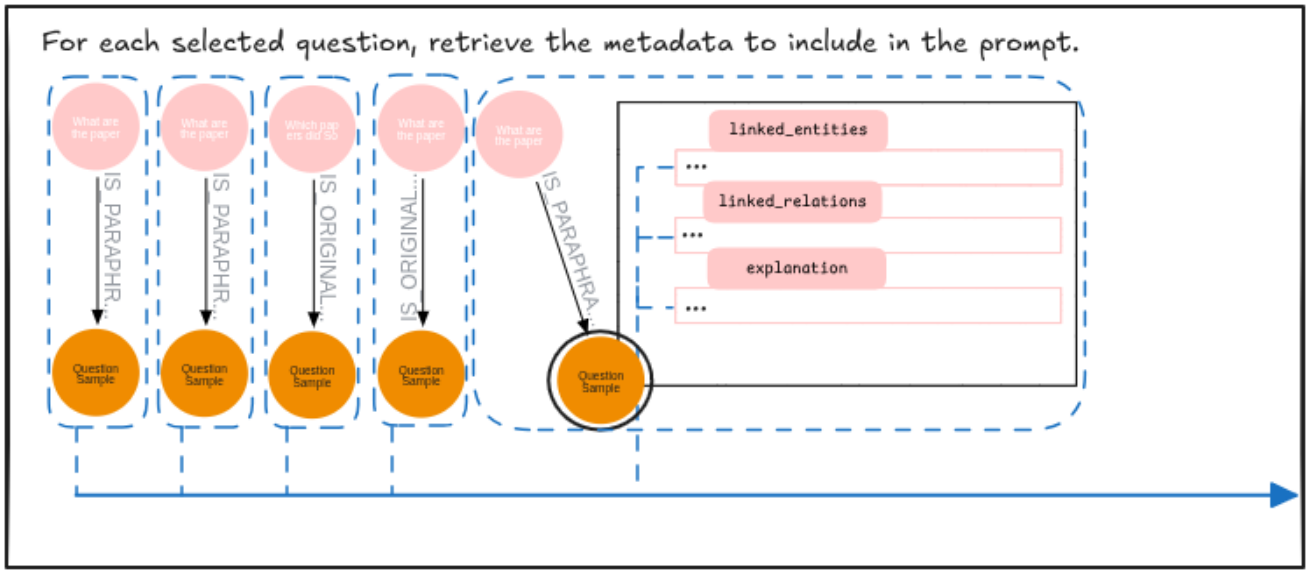

Figure 3.14: *For each retained sample, the metadata stored on the connected* `QuestionSample` *node are collected and assembled into the same format used by the static few-shot examples.*

The retrieved examples are appended to the prompt as a sequence of alternating human and ai messages, in the same form described in section 3.2.5.2 for the static examples and immediately after them when both techniques are enabled. The user question with its linked entities and linked relations follows the dynamic block as the last human message of the prompt, and the LLM is invoked under the structured output schema described in section 3.2.5.4. The actual contribution of Dynamic Few-Shot, of the Filter by Query Type, and of their interaction with the value of k is quantified by the ablation study presented in chapter 4.

The figure below summarizes the full prompt construction and inference flow of the Query Generator across the techniques described in the previous subsections.

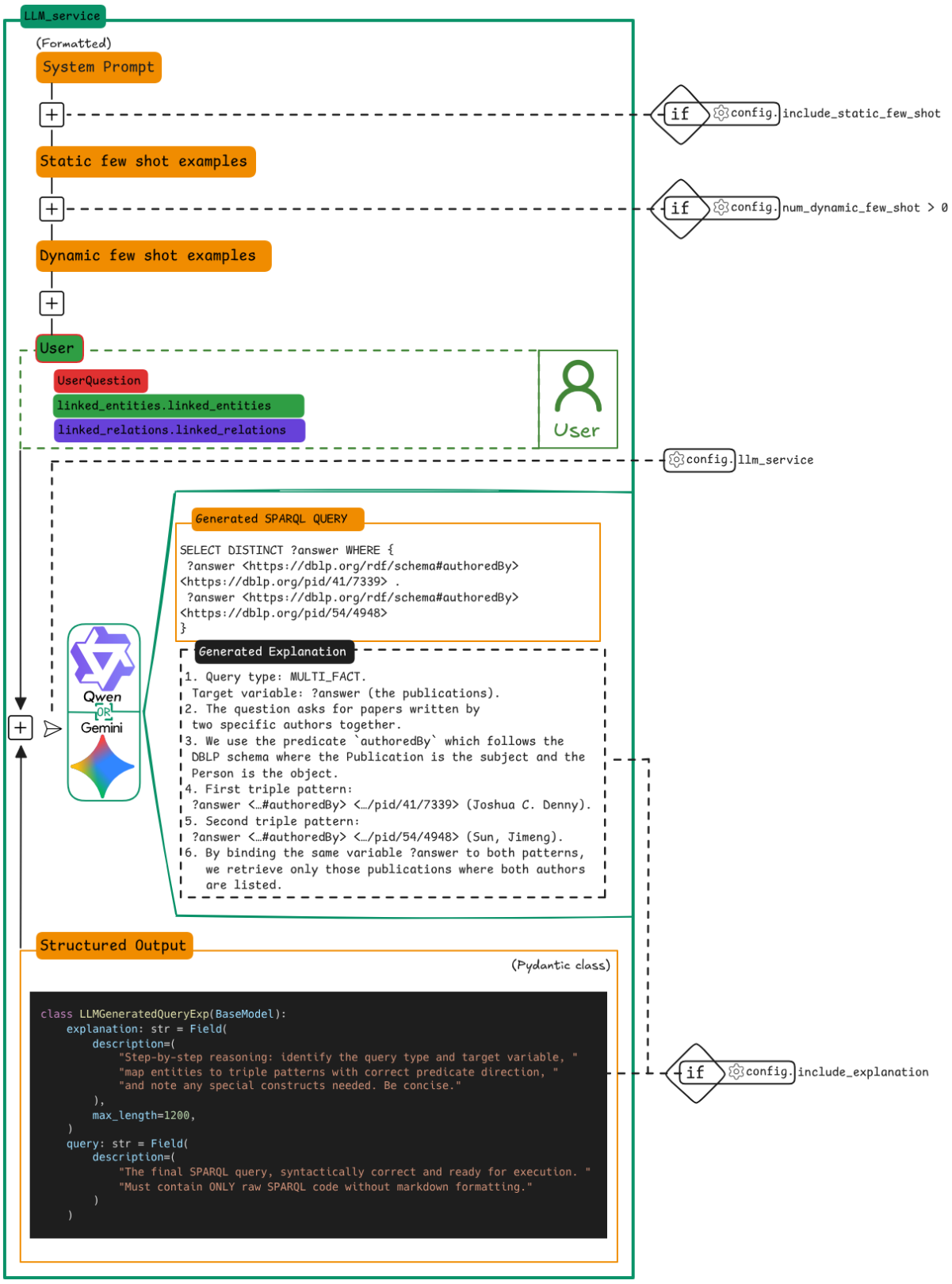

Figure 3.15: *Full prompt construction and inference flow of the Query Generator. The system prompt and the user message are always present; the static and the dynamic few-shot blocks are inserted only when their configuration flag is enabled; the explanation field is included in the structured output schema only when the explanation flag is enabled.*

3.2.6 Query Executor

The Query Executor takes the SPARQL query produced by the Query Generator and runs it against the public dblp SPARQL endpoint [1] at `https://sparql.dblp.org/sparql`. The endpoint returns the result of the query in the SPARQL 1.1 Query Results JSON Format[4], which the Query Executor returns as a `SparqlResult` object, the final output of the pipeline.

[4]https://www.w3.org/TR/sparql11-results-json/

3.2.7 Demo

The demo is a web application that runs the pipeline on a user question one stage at a time, and that lets the user review and edit the partial output of each stage before it is passed to the next. The pipeline used by the demo is the one declared in `demo.yml`, which selects `LLMInfoExtractor` and `ApiEntityLinker` as the upstream modules, so that any natural-language question can be answered and not only the questions of DBLP-QuAD.

The interactive flow is built on top of `run_iter`, the variant of the pipeline already mentioned in section 3.2.1 that yields control after every stage. After each yield, the partial output is displayed with edit widgets, and the iterator is resumed only when the user clicks Continue. This adds a human-in-the-loop layer on top of the pipeline, so that a mistake in an early stage can be corrected before it propagates to the next.The figures below illustrate the flow on a single example question.

The user enters the question in the input area and starts the run.

Figure 3.16: *Question entry. The user types a natural-language question and clicks Run pipeline to start the run.*

The Information Extractor displays the predicted query type in a drop-down and the named entities in a table. The user can change the query type and can add, remove, or rename entities before continuing.

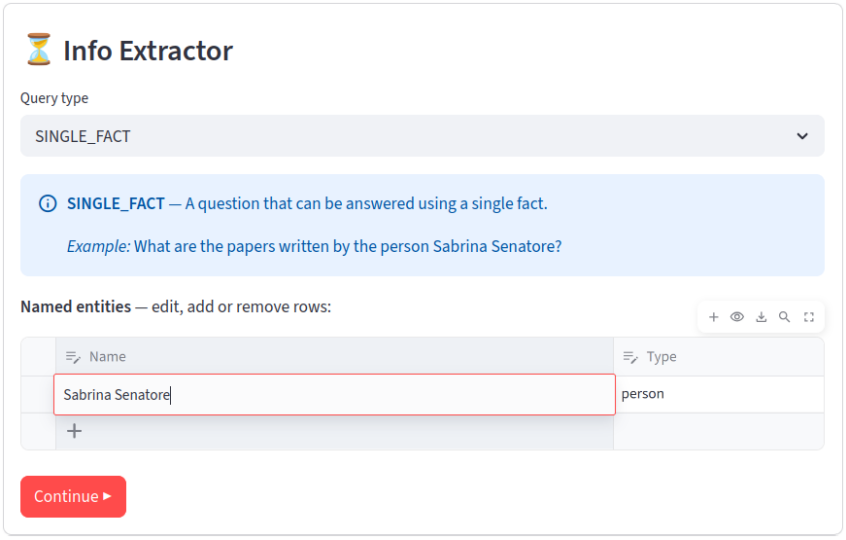

Figure 3.17: *Information Extractor stage. The predicted query type is shown in a drop-down with a short description and a sample question; the named entities are listed in an editable table.*

The Entity Linker displays the URI assigned to each named entity by the dblp search API. The user can correct a wrong URI, replace it with another, or remove a candidate that the linker should not have included.

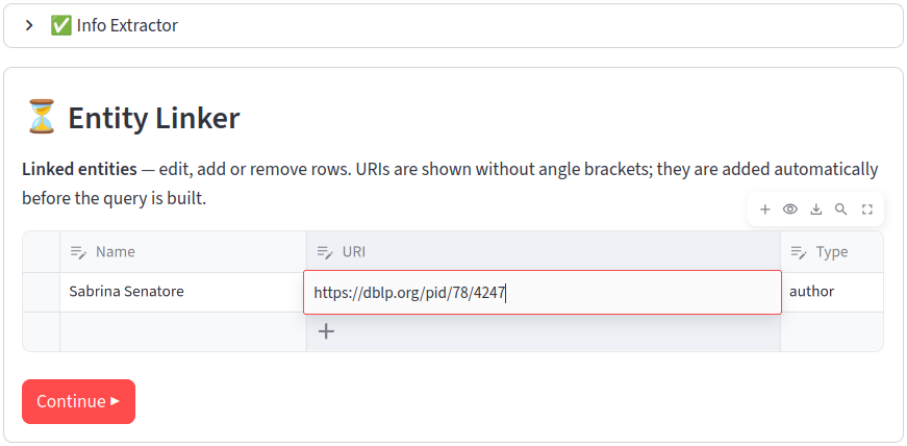

Figure 3.18: *Entity Linker stage. The URI assigned to each named entity is shown next to its name; the user can edit, add, or remove rows before continuing.*

The Relation Linker displays the full DBLP ontology, with the predicates selected by the linker already ticked. The user can tick further predicates that the linker missed, or untick predicates that are not relevant to the question, before continuing.

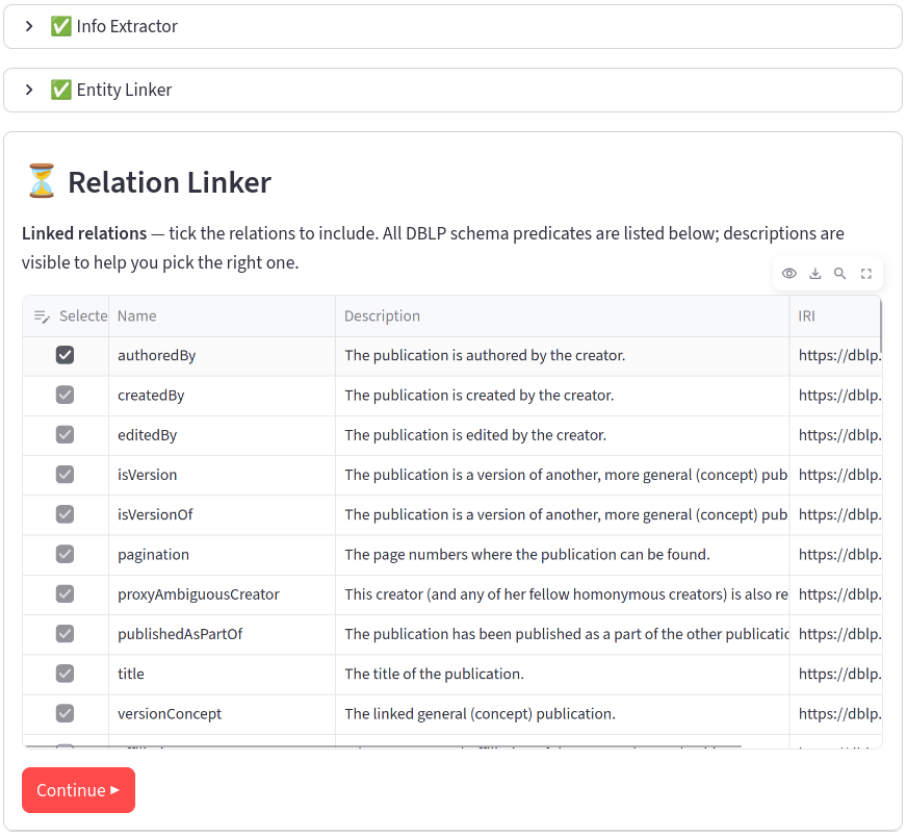

Figure 3.19: *Relation Linker stage. The full DBLP ontology is listed in a table; the predicates selected by the linker are pre-ticked, and the user can change the selection before continuing.*

After the Relation Linker, the Query Generator and the Query Executor are run together and their result appears in a single panel. The generated SPARQL query is shown in an editor with syntax highlighting, and the result of its execution on the dblp endpoint is shown below it. If the result is not what the user expected, the query can be edited in the editor and re-run with the Run query button, without re-running the upstream stages: small mistakes such as a misspelled predicate or a wrong URI can be fixed and tested directly on the query.

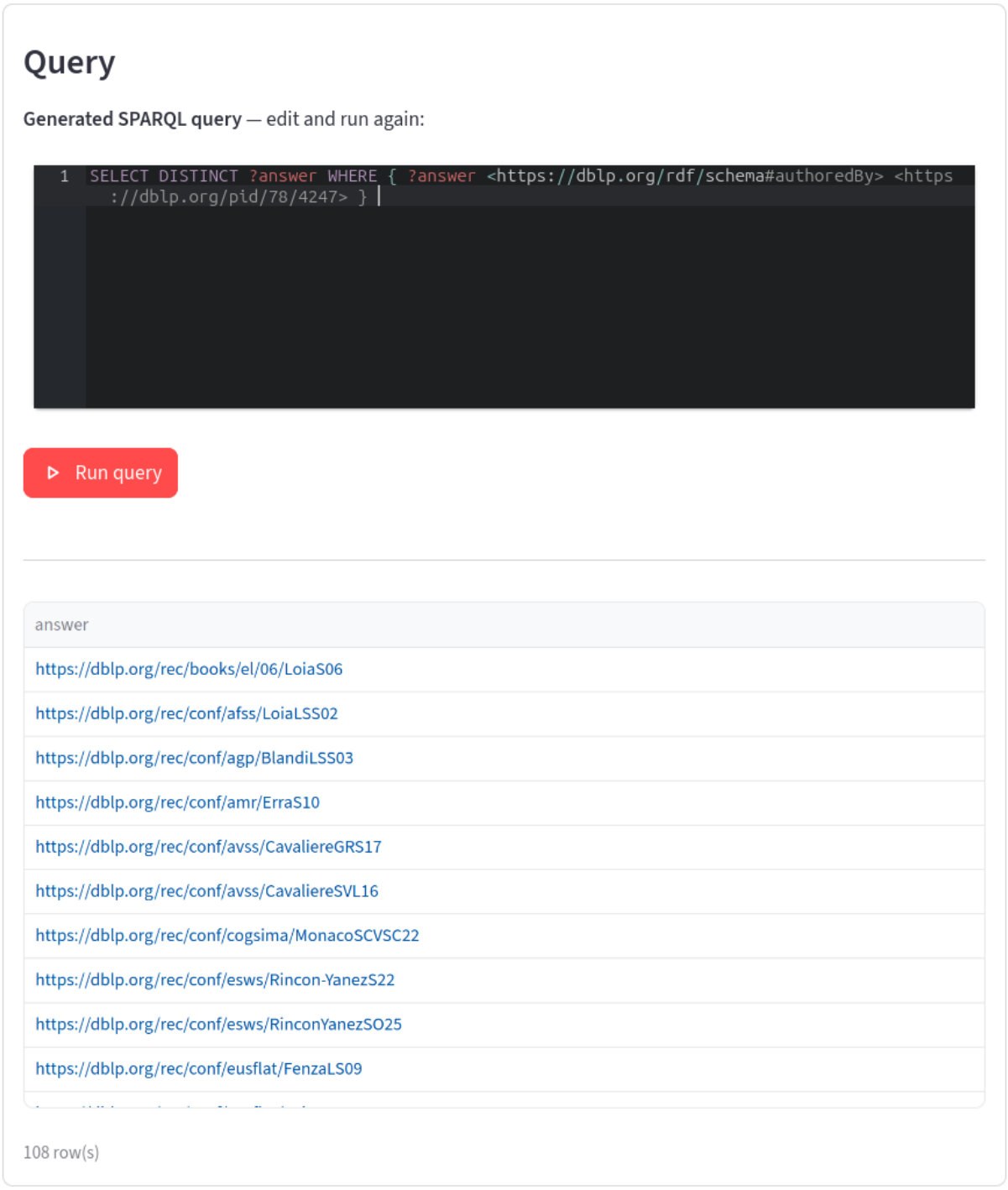

Figure 3.20: *Query panel. The generated SPARQL query is shown in an editor and its result on the dblp endpoint is shown below; the query can be edited and re-run without re-running the upstream stages.*

3.3 Tools, Technologies and Models used for Implementation

3.3.1 Development Environment

The system is implemented in Python 3.12 and managed with uv[5], a package manager that reads the project dependencies declared in `pyproject.toml`, pins them in a lockfile (`uv.lock`), and builds the project virtual environment in a single tool. The lockfile is more robust than a plain `requirements.txt` file because it pins every transitive dependency to an exact version with the corresponding hash, and the resolver is faster than the classic pip workflow. As a result, the dependency tree can be reproduced on a fresh machine with a single `uv sync` command.

The project can be used in two modes, both built from the same multi-stage Dockerfile and orchestrated through `docker-compose.yml`, which composes the application container together with the Neo4j database described in section 3.3.4. The developer mode is meant to reproduce the experiments of chapter 4 and to modify the implementation: a second compose file in `.devcontainer/docker-compose.yml` is merged on top of the main one to install the development dependencies and to bind-mount the project sources from the host, so that edits are picked up live without rebuilding the image. The same environment can also be entered from VS Code through the DevContainers[6] specification, in order to use the same VS Code settings and extensions adopted during the development of the system. The user mode builds the image with the runtime dependencies only and is used to launch the demo of section 3.2.7.

Both modes rely on the same `services.yml`, but on a distinct pipeline configuration: `experiment.yml` for the developer mode and `demo.yml` for the user mode. The two files differ, for example, in the choice of the Information Extractor of section 3.2.2 and the Entity Linker of section 3.2.3: `experiment.yml` selects the gold variants, while `demo.yml` selects `LLMInfoExtractor` and `ApiEntityLinker`.

[5] https://docs.astral.sh/uv/
[6] https://containers.dev/

3.3.2 Software Libraries

Four software libraries are used in the application code; the others are tied to specific external technologies and appear in the next subsections.

Pydantic[7] is a data validation library based on Python type hints, used in the system together with its `pydantic-settings` companion. It plays the following roles:

- It defines and validates the YAML configuration files of section 3.2.1: `services.yml` is read into a `ServiceRegistryConfig` instance, while the pipeline configuration file (`experiment.yml` or `demo.yml`) is read into a `PipelineConfig` instance.
- It implements the discriminated-union mechanism that drives the service and module factories of section 3.2.1: every configuration class carries a literal field (`type` or `strategy`) annotated as the discriminator of a union of classes, and the corresponding factory dispatches on the value of this field to build the right one.
- It defines the output contracts that the modules of the pipeline pass to one another (`ExtractedInfo` , `LinkedEntities` , `LinkedRelation` , `GeneratedQuery` , `SparqlResult`), accumulated into `PipelineOutput` as the question moves through the stages.
- It defines the structured output schemas that the LLM is asked to fill in the Information Extractor of section 3.2.2 and in the Query Generator of section 3.2.5.4, accepted natively by the LLM libraries.
- It loads the secrets of the application (Google API keys, Neo4j credentials, RunPod token) from a `.env` file through its `BaseSettings` class.

pandas[8] is used by the experiment runner of chapter 4 to compute the metrics of the ablation study and to produce the evaluation tables.

SPARQLWrapper[9] is the SPARQL client used by the Query Executor of section 3.2.6 to send the generated query to the dblp endpoint and to parse its response.

Streamlit[10] is a framework that allows building web applications in Python, and it was used to implement the demo of section 3.2.7. The SPARQL editor of the query panel is provided by streamlit-ace[11], a Streamlit component that wraps the Ace editor with syntax highlighting.

[7] https://github.com/pydantic/pydantic
[8] https://pandas.pydata.org/
[9] https://sparqlwrapper.readthedocs.io/
[10] https://streamlit.io/
[11] https://github.com/okld/streamlit-ace

3.3.3 Language Models and Inference

The system relies on LangChain[12], a Python framework that provides a common interface for accessing several LLM and embedding providers, so that `LLMService` and `EmbeddingService` of section 3.2.1 can support their backends through a single internal interface.

Three Google models are configured in the system, accessed through the `langchain-google-genai` package of LangChain; the system can be configured to call them either through the Google AI Studio API or through the Vertex AI API.
Gemini 3.1 Flash Lite[13] is the cloud LLM of the ablation study of chapter 4. Its thinking level is set to `minimal`, the lowest level exposed by the API: according to the official documentation[14], `minimal` corresponds to no reasoning for the majority of queries, but does not guarantee that reasoning is disabled. Gemini 3 Flash[15] with thinking level set to `high` is the teacher of the synthetic explanations of section 3.2.5.3. The embedding model `gemini-embedding-001`[16] is used by the RAG relation linker of section 3.2.4, with task type `RETRIEVAL_DOCUMENT` on the property descriptions and `RETRIEVAL_QUERY` on the user question, and by the dynamic few-shot retrieval in the Query Generator of section 3.2.5.5, with task type `SEMANTIC_SIMILARITY`.

Qwen 3.5 4B UD-Q4_K_XL[17], the local LLM of the ablation study of chapter 4, is a 4 billion parameters model in a 4-bit quantization variant distributed by Unsloth. The model is served through llama.cpp[18], an open-source inference engine that exposes an OpenAI-compatible HTTP API, accessed by the system through the `langchain-openai` package of LangChain. llama.cpp can be launched either locally, as a separate Docker container with two profiles for CPU and CUDA inference, or remotely on a GPU rented from RunPod[19]; in the latter case, a small command-line script reads the configuration from `services.yml`, starts the pod, and returns the URL of the created pod.

[12] https://www.langchain.com/
[13] https://deepmind.google/models/gemini/flash-lite/
[14] https://ai.google.dev/gemini-api/docs/thinking#thinking-levels
[15] https://deepmind.google/models/gemini/flash/
[16] https://ai.google.dev/gemini-api/docs/embeddings
[17] https://unsloth.ai/docs/models/qwen3.5
[18] https://github.com/ggml-org/llama.cpp
[19] https://www.runpod.io/

3.3.4 Knowledge Graph Backend

Neo4j[20] 5.26 Community Edition is the graph database used by `Neo4jService` of section 3.2.1, accessed through the official `neo4j` Python driver[21]. A single Neo4j instance hosts the two subgraphs that the system relies on: the DBLP ontology used by the RAG relation linker of section 3.2.4, and the DBLP-QuAD training set used by the dynamic few-shot retrieval in the Query Generator of section 3.2.5.5. The queries against both subgraphs are written in Cypher, the native query language of Neo4j, and the embeddings stored as vector properties of the nodes are searched through the native `vector.similarity.cosine` function.

Two plugins are configured on top of Neo4j. APOC[22] (Awesome Procedures On Cypher) is a general-purpose extension library, used in the system for the bulk import of the DBLP-QuAD samples and their embeddings from JSON files through the `apoc.load.json` procedure. neosemantics (n10s)[23] is the RDF and semantics plugin of Neo4j, used in the system for the import of the DBLP ontology from its Turtle file through the `n10s.onto.import.fetch` procedure.

[20] https://neo4j.com/
[21] https://neo4j.com/docs/python-manual/current/
[22] https://neo4j.com/labs/apoc/
[23] https://neo4j.com/labs/neosemantics/

3.3.5 Hardware

The hardware target of the system is the laptop on which the project has been developed: a machine with an Intel i7-12700H CPU, 32 GB of RAM, and an NVIDIA RTX 3050 Ti GPU with 4 GB of VRAM. Qwen 3.5 4B UD-Q4_K_XL has been chosen as the local LLM because it is the most capable Qwen variant that runs on this hardware: the model requires around 5 GB of VRAM, so part of the layers is offloaded to the CPU through llama.cpp, but the inference latency remains within acceptable limits for the demo of section 3.2.7.

The same model has been evaluated in the ablation study of chapter 4 on different hardware: an NVIDIA RTX 3090 with 24 GB of VRAM rented from RunPod, on which all the layers fit on the GPU and the inference is fast enough to run the ablation in a reasonable time. The choice does not affect the quality of the generated queries, since the model and its sampling parameters are the same in both setups, and the GPU only changes the speed at which inference is performed.

4 Experimental Validation and Applicative Aspects

4.1 Description of Data used for Experimentation

The experimental validation uses the DBLP-QuAD dataset[3] and a local SPARQL endpoint loaded with the dblp RDF dump on which the dataset was built.

4.1.1 The DBLP-QuAD Dataset

DBLP-QuAD is a dataset of 10,000 question-SPARQL pairs over the dblp scholarly knowledge graph, split into 7000 training, 1000 validation, and 2000 test instances. The questions are generated from 98 hand-written SPARQL templates with placeholders for entity names and literal values, and the placeholders are filled with values taken from a random portion of the dblp KG.

Each instance contains:

- a SPARQL query against the dblp KG;
- a main natural-language question and a paraphrase of it;
- the gold linked entities and predicates that appear in the SPARQL query;
- one of the ten query types described in section 3.2.2.1;
- two boolean flags that indicate whether the question is temporal and whether the SPARQL template is kept out of the training split.

As an example, the question `Q0203` appears in the dataset as


```
{
  "id": "Q0203",
  "query_type": "DOUBLE_INTENT",
  "question": {
    "string": "List the venues in which Matt Rowe published papers in the last
    ↪  six years and the titles of these papers."
  },
  "paraphrased_question": {
    "string": "List the titles of the papers that Rowe, M. published in the
    ↪  last six years and in which venues."
  },
```

```
  "query": {
    "sparql": "SELECT DISTINCT ?firstanswer ?secondanswer WHERE { ?x
    ↪  <https://dblp.org/rdf/schema#authoredBy>
    ↪  <https://dblp.org/pid/247/9429> . ?x
    ↪  <https://dblp.org/rdf/schema#yearOfPublication> ?y . FILTER(?y >
    ↪  YEAR(NOW())-6) . ?x <https://dblp.org/rdf/schema#publishedIn>
    ↪  ?firstanswer . ?x <https://dblp.org/rdf/schema#title> ?secondanswer }"
  },
  "template_id": "TC25",
  "entities": ["<https://dblp.org/pid/247/9429>"],
  "relations": [
    "<https://dblp.org/rdf/schema#authoredBy>",
    "<https://dblp.org/rdf/schema#yearOfPublication>",
    "<https://dblp.org/rdf/schema#publishedIn>",
    "<https://dblp.org/rdf/schema#title>"
  ],
  "temporal": false,
  "held_out": false
}
```

A separate file stores the gold answer of every SPARQL query. For `Q0387` (“Did the author Beverley Gable publish the paper ‘Listening in a second language’?”), a BOOLEAN question, this is the ASK result

```
{
  "head": { "link": [] },
  "boolean": true
}
```

For `Q0203`, instead, it is the SELECT result

```
{
  "head": { "link": [], "vars": ["firstanswer", "secondanswer"] },
  "results": {
    "distinct": false,
    "ordered": true,
    "bindings": [
      {
        "firstanswer": {
          "type": "literal",
          "value": "DART/MIL3ID@MICCAI"
        },
        "secondanswer": {
          "type": "literal",
          "value": "Few-Shot Learning with Deep Triplet Networks for Brain
          ↪  Imaging Modality Recognition."
```

```
        }
      },
      {
        "firstanswer": {
          "type": "literal",
          "value": "CoRR"
        },
        "secondanswer": {
          "type": "literal",
          "value": "Few-shot Learning with Deep Triplet Networks for Brain
          ↪  Imaging Modality Recognition."
        }
      }
    ]
  }
}
```


To test the generalization of systems trained or prompted on the training split, twenty templates and two question forms are kept out of it. As a result, the test split contains three kinds of questions [3]: 81.2% are in-distribution, where the template of the question appears in the training split; 15.1% are compositional, where the question combines patterns of the training split in new ways; and 3.8% are zero-shot, where the template of the question never appears in the training split. The test split is also balanced across the ten query types, with 200 questions per type.

4.1.2 SPARQL Endpoint

The SPARQL endpoint used for evaluation is a local docker instance of Virtuoso, loaded with the dblp RDF dump archived on Zenodo[1]. The dump is the version of the dblp knowledge graph used to compute the gold answers of DBLP-QuAD. The public dblp endpoint cannot be used instead, because the dblp knowledge graph has changed since 2022.

[1] https://zenodo.org/records/7638511

4.2 Description of Result Evaluation Metrics

The evaluation compares the pipeline answer on a test question with the gold answer recorded in DBLP-QuAD for the same question. The pipeline answer is the `SparqlResult` returned by the Query Executor of section 3.2.6. Both answers follow the standard SPARQL JSON result format already illustrated in section 4.1.1: a boolean for `ASK` queries, and a structured object with variable bindings for `SELECT` queries.

4.2.1 Answer Extraction

Before any metric can be computed, each `SparqlResult` is flattened into a list of values. For an `ASK` result, the flattening returns the boolean as a one-element list, `["true"]` or `["false"]`. For a `SELECT` result, the flattening collects every bound value from every row into a single flat list, without distinguishing variable names: two variables in the same row contribute two separate entries.

As an example, the gold answer of `Q0203` shown in section 4.1.1 is flattened into

```
[
  "DART/MIL3ID@MICCAI",
  "Few-Shot Learning with Deep Triplet Networks for Brain Imaging Modality
   ↪ Recognition.",
  "CoRR",
  "Few-shot Learning with Deep Triplet Networks for Brain Imaging Modality
   ↪ Recognition."
]
```

This procedure reproduces the input format expected by the `qa_el_eval.py` script of the Scholarly QALD challenge[2], from which the micro F_1 metric of section 4.2.3 is taken.

4.2.2 Per-Query Metrics

The per-query metrics treat the two flat lists of values as sets and measure their overlap. We denote by G the set of gold values for a fixed question and by S the set of system values produced for the same question.

[2]https://github.com/debayan/scholarly-QALD-challenge/blob/main/2023/datasets/dblp-kgqa/scripts/qa_el_eval.py

Every per-query metric is computed from three values: the number of true positives (TP), the number of false positives (FP), and the number of false negatives (FN):

$$TP = |G \cap S| \qquad FP = |S \setminus G| \qquad FN = |G \setminus S| \tag{4.1}$$

From these, precision (P), recall (R), and F_1 score follow their usual definitions:

$$P = \frac{TP}{TP + FP} \qquad R = \frac{TP}{TP + FN} \qquad F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{4.2}$$

When a denominator is zero, the corresponding metric is set to zero.

Two further per-query indicators are recorded together with P, R, and F_1. The Exact Match indicator EM is true when both FP and FN are zero, that is, when the system answer matches the gold answer exactly:

$$EM = \mathbf{1}[\, FP = 0 \,\wedge\, FN = 0 \,] \tag{4.3}$$

The Empty indicator is true when the system list is empty, regardless of whether the gold list is empty:

$$Empty = \mathbf{1}[\, |S| = 0 \,] \tag{4.4}$$

The Empty indicator is used as a diagnostic signal: when its count is high in a group of questions, it tells us that the model fails to produce any answer on part of the test split, rather than producing wrong answers.

4.2.3 Aggregate Metrics

The aggregate metrics are calculated from the per-query precision, recall, and F_1 over a group of questions, where the group is either the entire test split or the subset of questions of a single query type.

The macro F_1 is the primary metric used in the ablation. It is the mean of the per-query F_1 scores:

$$F_1^{macro} = \frac{1}{N} \sum_{i=1}^{N} F_1^{(i)} \tag{4.5}$$

where N is the number of questions in the group and $F_1^{(i)}$ is the per-query F_1 of the i-th question, computed from equation (4.2).

The micro F_1 reproduces the metric of the `qa_el_eval.py` script of the Scholarly QALD challenge. It first sums the per-query counts over the group:

$$TP^{micro} = \sum_{i=1}^{N} TP^{(i)} \qquad FP^{micro} = \sum_{i=1}^{N} FP^{(i)} \qquad FN^{micro} = \sum_{i=1}^{N} FN^{(i)} \tag{4.6}$$

then computes precision, recall, and F_1 from the global counts using the formulas of equation (4.2). Compared to macro F_1, micro F_1 weights every value equally, so a question with many values in its gold answer contributes more to the score than a question with a single value.

The Exact Match is aggregated as the mean of the per-query EM across the group.

The Empty count is aggregated as the sum of the per-query $Empty$.

4.2.4 DBLP-QuAD F1

In addition to the standard metrics, we report a third aggregate metric, the DBLP-QuAD F_1, derived from the `evaluate_dblp.py` script that comes with the dataset[3].

For an `ASK` result, the metric uses the same answer-extraction procedure as in section 4.2.1. For a `SELECT` result, it differs from the standard ones in three ways.

The first difference is the answer-extraction procedure: the metric does not flatten every bound value, but picks the answer values based on the variable names declared in the head of the result:

[3] https://github.com/awalesushil/DBLP-QuAD/blob/main/Query2Question/src/evaluate.py

- when the head declares a single answer variable (`?answer` or `?count`), the procedure returns the list of values bound to that variable;
- when the head declares both `?firstanswer` and `?secondanswer`, the procedure pairs the two values of each row and returns a list of pairs.

The pairing is needed for `DOUBLE_INTENT` queries. For example, on the question "Mention the papers published by Wazir Muhammad and in which year", the gold rows are of the form $(paper, year)$, where each paper is paired with its own year of publication. The pairing keeps this association in the extracted list, so that a row from the system is counted as correct only when both its values match the gold row. If the system returns the right papers but assigns the wrong year to one of them, for example $(paper_A, 2019)$ instead of the gold $(paper_A, 2020)$, the resulting row does not match the gold row and is counted as an error.

The second difference is the way the per-query precision and recall are computed: the numerator is the same as in equation (4.2), but the denominators are the lengths of the system list (n_i^S) and of the gold list (n_i^G):

$$P_{dblp}^{(i)} = \frac{TP_i}{n_i^S} \qquad R_{dblp}^{(i)} = \frac{TP_i}{n_i^G} \tag{4.7}$$

Unlike the macro F_1 of equation (4.5), which treats both answers as sets, the DBLP-QuAD F_1 keeps duplicate values in these lengths.

The third difference is the way the global F_1 is computed. The per-query precision and recall are first averaged over the group:

$$P_{dblp} = \frac{1}{N}\sum_{i=1}^{N} P_{dblp}^{(i)} \qquad R_{dblp} = \frac{1}{N}\sum_{i=1}^{N} R_{dblp}^{(i)} \tag{4.8}$$

The DBLP-QuAD F_1 is then computed by applying the F_1 formula of equation (4.2) to these two averages, rather than averaging the per-query F_1 scores:

$$F_1^{dblp} = \frac{2 \cdot P_{dblp} \cdot R_{dblp}}{P_{dblp} + R_{dblp}} \tag{4.9}$$

4.3 Definition of the Experimental or Verification Protocol

The experimental work is organized around four questions (Q1, Q2, and Q3 correspond to the three areas of advancement identified at the end of chapter 2):

- Q0: is the best configuration of the system competitive with the existing KGQA systems on DBLP-QuAD?
- Q1: what is the contribution of each prompting technique (static and dynamic few-shot examples, filter by query type, and chain-of-thought) to the quality of the generated SPARQL queries, and how does this contribution change between gemini-3.1-flash-lite and qwen-4B?
- Q2: does dynamic relation linking, based on retrieval-augmented generation over the DBLP ontology, preserve the answer accuracy while reducing the size of the prompt compared to the full-schema baseline?
- Q3: how competitive is Qwen compared to Gemini on the same pipeline, and where does the gap concentrate across query types?

4.3.1 Experiment Runner

To answer these questions, the system is run several times on the DBLP-QuAD test split, each time under a different pipeline configuration. Every run is executed by the `experiment/main.py` script. The script first reads the service-registry configuration from `config/services.yml` and the pipeline configuration from `config/experiment.yml`. It then loads the services into the registry, builds the pipeline through the `PipelineFactory`, and loads the test split of DBLP-QuAD.
For every question of the split, the pipeline produces a `PipelineOutput` instance with the extracted information, the linked entities, the linked relation, the generated query, and the SPARQL result. This output is wrapped into an `ExpResult`, which adds the question identifier, and appended to the `ExpResults` collection of the run.

A logging system records what each module of the pipeline produces on every sample (including details not kept in `ExpResults`, such as the rank of the retrieved few-shot examples or the chain-of-thought explanation) and the errors raised during the run. A failure on a single sample is logged and the run continues with the next question.

Since each run is long, the script implements a few safety mechanisms, such as periodic checkpoints of the partial `ExpResults` to disk and automatic retries on the cloud LLM calls.

When the run completes, the metrics of section 4.2 are computed and saved together with the results file (`exp_results.json`), the configurations used (`experiment.yml` and `services.yml`), and the log (`experiment.log`).

Note: across all the runs of this chapter, the Information Extractor and the Entity Linker are held fixed at their gold variants `GoldInfoExtractor` and `GoldEntityLinker`, as anticipated in chapter 3.

4.3.2 Comparison with the State of the Art Setup

To answer Q0, the best configuration of the system is compared to three KGQA systems already presented in chapter 2: the T5-Base baseline of [3], NLQxform [27], and ASK-DBLP [28]. The three systems do not all publish the same F_1 metric, so the comparison requires some clarification.

NLQxform reports the F_1 of the `qa_el_eval.py` script of the Scholarly QALD Challenge, which is the micro F_1 of section 4.2.3.
The T5 baseline reports an F_1 score whose definition is not given in the paper; the most natural match is the DBLP-QuAD F_1 of section 4.2.4, computed by the `evaluate_dblp.py` script released together with the dataset.
ASK-DBLP does not specify its evaluation procedure either, and presents its score next to the T5 baseline in a single comparison table, so the same DBLP-QuAD F_1 is assumed for both. For our system we report both F_1 variants alongside the published scores, so that the comparison with each one is between values computed by the same procedure.

4.3.3 Ablation Setup

The ablation varies three settings across runs: the Query Generator configuration, the Relation Linker strategy, and the LLM that generates the SPARQL query (Gemini or Qwen).

4.3.4 Query Generator Ablation

The Query Generator is evaluated under nine configurations, labelled with Roman numerals from I to IX. Each configuration combines the system prompt (SP) with a subset of the prompting techniques described in section 3.2.5: static few-shot (SFS), dynamic few-shot (DFS), filter by query type (FQT), and chain-of-thought (CoT). The full list is given in Table table 4.1, and the ablation is run on both LLMs.

Configuration	SP	SFS	DFS	FQT	CoT
I	•				
II	•				•
III	•	•			
IV	•	•			•
V	•		•		
VI	•		•	•	
VII	•		•		•
VIII	•		•	•	•
IX	•	•	•	•	•

Table 4.1: *The nine Query Generator configurations of the ablation. A bullet in a column means that the corresponding technique is enabled in that configuration.*

The configurations are designed to allow several direct comparisons:

- I and II compare the system-prompt-only baseline with and without CoT.
- III and V compare static and dynamic retrieval, both without CoT.
- IV and VII repeat the same comparison with CoT enabled. The pairs (IV, III) and (VII, V) show the gain of CoT on the static and on the dynamic configuration.
- VI adds the filter by query type on top of V.
- VIII is the full dynamic combination (DFS + FQT + CoT).
- IX adds the static few-shot to VIII, to test whether SFS adds value when DFS is already in the prompt.

In every run of this ablation, the Relation Linker is held fixed at the full-schema strategy of section 3.2.4.1. The configurations that use dynamic few-shot examples are run with k = 5 by default. To study how the size of the dynamic context affects the generated query, configuration VIII is also run at k = 3, k = 10, and k = 15. The value k = 20 is not tested because it would exceed the 16k context window of Qwen.

4.3.5 Relation Linker Ablation

The `RAGRelationLinker` is evaluated with the configurations X and XI, described in section 3.2.4.2, and is used instead of the `FullSchemaRelationLinker` baseline. Configuration X uses the relation linker without the domain-range flag, while configuration XI enables the flag. Both configurations are tested with the best Query Generator configuration for each LLM, where best is defined by the macro F_1 score on the test split

The Relation Linker uses the 2022 ontology on which the gold queries of DBLP-QuAD were built, so that the predicates available to the linker match those used in the gold queries.

4.4 Result Presentation and Analysis

4.4.1 Comparison with the State of the Art

The table below compares the best Gemini and Qwen configurations of our system with the published scores of T5-Base, NLQxform, and ASK-DBLP.

System	DBLP-QuAD F1	micro F1
T5-Base	0.868	-
NLQxform	-	0.849
ASK-DBLP	0.761	-
Our - gemini-3.1-flash-lite	0.933	0.990
Our - qwen-3.5-4B	0.851	0.970

Table 4.2: *Comparison with the three KGQA systems described in chapter 2.*

The best Gemini configuration is VIII (SP + DFS + FQT + CoT) with k = 10 dynamic few-shot examples and the full DBLP ontology in the prompt. The best Qwen configuration is X (SP + DFS + FQT + CoT) with k = 15 dynamic few-shot examples and the RAG relation linker retrieving the 20 most relevant properties.

T5-Base is fine-tuned on the DBLP-QuAD training set and receives gold entities and gold relations as input. Our Gemini gains 6.5% on DBLP-QuAD F1, while our Qwen loses 1.7%.

NLQxform fine-tunes a BART model on the training set and uses a template base built from the gold queries to correct its output. It performs its own entity linking through the DBLP search APIs, while relation linking is handled implicitly by the fine-tuned model. Our Gemini gains 14.1% on micro F1, and our Qwen gains 12.1%.

ASK-DBLP uses an LLM (Qwen 2.5 Coder 32B Instruct) without fine-tuning and retrieves dynamic few-shot examples from the training set, as our system does. It performs entity linking through DBLPLink2.0[29] and includes the full DBLP ontology in the prompt, the same setting as our full-schema relation linker. Our Gemini gains 17.2% on DBLP-QuAD F1, and our Qwen gains 9.0%.

4.4.2 Query Generator Ablation

4.4.2.1 Aggregate Results

The two tables below report the aggregate metrics for each configuration of table 4.1 on Gemini and on Qwen, with k = 5 dynamic few-shot examples in every configuration that uses them.

Config	EM	macro F1	Empty	micro F1	DBLP F1
I	62.9%	0.675	305	0.708	0.660
II	61.2%	0.653	329	0.766	0.637
III	70.1%	0.757	182	0.765	0.708
IV	68.0%	0.729	205	0.752	0.701
V	87.5%	0.898	84	0.966	0.882
VI	89.5%	0.915	76	0.969	0.908
VII	87.7%	0.902	47	0.974	0.892
VIII	90.1%	0.922	41	0.973	0.919
IX	90.5%	0.927	45	0.974	0.923

Table 4.3: *Aggregate metrics on the test split for each Query Generator configuration on Gemini.*

Config	EM	macro F1	Empty	micro F1	DBLP F1
I	27.9%	0.288	939	0.038	0.250
II	37.9%	0.392	863	0.118	0.352
III	32.5%	0.331	870	0.126	0.306
IV	46.9%	0.492	613	0.233	0.455
V	71.4%	0.726	259	0.872	0.702
VI	77.2%	0.790	213	0.853	0.778
VII	79.2%	0.809	182	0.932	0.797
VIII	79.6%	0.813	182	0.963	0.812
IX	81.9%	0.837	144	0.946	0.833

Table 4.4: *Aggregate metrics on the test split for each Query Generator configuration on Qwen.*

With only the system prompt (configuration I), Gemini reaches a macro F1 of 0.675 and Qwen 0.288, with 305 and 939 empty answers respectively. Qwen fails to produce any answer for almost half of the test split. The system prompt alone is not enough on either LLM.

4.4.2.2 Effect of Dynamic Few-Shot

Four pairs of configurations isolate the effect of DFS: I (SP) against V (SP + DFS), II (SP + CoT) against VII (SP + DFS + CoT), III (SP + SFS) against V, and IV (SP + SFS + CoT) against

VII. The table below reports the macro F1 of each side of the pair and the change introduced by DFS.

Pair	Base prompt	Gem F1	Δ Gem	Qwen F1	Δ Qwen
I → V	SP	0.675 → 0.898	+22.3%	0.288 → 0.726	+43.8%
II → VII	SP + CoT	0.653 → 0.902	+24.9%	0.392 → 0.809	+41.7%
III → V	SP + SFS	0.757 → 0.898	+14.1%	0.331 → 0.726	+39.5%
IV → VII	SP + SFS + CoT	0.729 → 0.902	+17.3%	0.492 → 0.809	+31.7%

Table 4.5: *Macro F1 of the base and DFS-active configurations on the four pairs, with the change in percentage points. Rows are sorted by Δ Qwen.*

The empty count also drops sharply on all pairs, e.g., from III to V the count drops from 182 to 84 on Gemini and from 870 to 259 on Qwen.

DFS is the technique that contributes most to performance on both LLMs: every DFS configuration (V to IX) scores higher than every non-DFS one (I to IV). The other techniques (SFS, FQT, CoT) also improve performance, but with smaller gains; they are discussed in the following subsections.

4.4.2.3 Effect of Filter by Query Type

The pair V against VI isolates the effect of FQT at k = 5. On Gemini, macro F1 grows from 0.898 to 0.915 (+1.7%). On Qwen, the gain is larger: from 0.726 to 0.790 (+6.4%).

The aggregate gain hides different effects across query types. The table below shows the per-query-type macro F1 for the V against VI pair on both LLMs.

Query type	GemV	GemVI	Δ Gem	QwenV	QwenVI	Δ Qwen
Double_negation	0.910	0.985	+7.5%	0.530	0.845	+31.5%
Union	0.805	0.799	-0.6%	0.478	0.707	+22.9%
Negation	0.940	1.000	+6.0%	0.865	1.000	+13.5%
Disambiguation	0.774	0.793	+1.9%	0.681	0.723	+4.2%
Single_fact	0.990	0.995	+0.5%	0.955	0.985	+3.0%
Count	0.890	0.875	-1.5%	0.675	0.680	+0.5%
Boolean	0.950	0.955	+0.5%	0.835	0.820	-1.5%
Multi_fact	0.914	0.930	+1.6%	0.776	0.755	-2.1%
Superl.+Comp.	0.832	0.836	+0.4%	0.653	0.629	-2.4%
Double_intent	0.972	0.979	+0.7%	0.814	0.759	-5.5%
TOTAL	0.898	0.915	+1.7%	0.726	0.790	+6.4%

Table 4.6: *Per-query-type macro F1 for V against VI on both LLMs. The Δ columns show the change in macro F1. Rows are sorted by Δ Qwen*

The largest gains appear on three query types: DOUBLE_NEGATION, UNION, and NEGATION. For each of them, FQT prevents the model from copying the structure of a different query type into the generated query.

The clearest case is DOUBLE_NEGATION on Qwen (+31.5%). In DBLP-QuAD, a DOUBLE_NEGATION question of the form "Was X not not Y?" cancels the two negations, and its gold query is a plain ASK on a single triple. A NEGATION question of the form "Was X not Y?" wraps the same triple inside a FILTER NOT EXISTS clause: the ASK returns True only when the triple is absent from the graph. The two types share similar wording and are easy to confuse. Consider Q0537, "Was the paper 'The Discrete Analytical Hyperspheres' not not published by Eric Andres?". The gold query is

```
ASK { <paper> <authoredBy> <author> }
```

For this question, Qwen V retrieves five dynamic examples: three NEGATION and two DOUBLE_NEGATION. The model imitates the NEGATION pattern and produces

```
ASK { <paper> <authoredBy> <author>
        FILTER NOT EXISTS { <paper> <authoredBy> <author> } }
```

The triple exists in the graph, but the FILTER NOT EXISTS clause requires the same triple to be absent. The two requirements cannot both hold, so the ASK returns False, the wrong answer. With FQT, the retrieval brings only DOUBLE_NEGATION examples, none of which contains FILTER NOT EXISTS. The model writes the plain ASK above and the answer is correct.

UNION on Qwen jumps by +22.9% for a similar reason. UNION questions are the only ones in the dataset whose gold query joins two graph patterns with the UNION keyword. Without FQT, the retrieval mixes UNION with other types and the model often writes a conjunction instead, asking for results that match both patterns at the same time. Take Q0649, "D. Eppstein and Liu, X. published which papers?". The gold query is

```
SELECT DISTINCT ?answer WHERE {
    { ?answer <authoredBy> <eppstein> }
    UNION
    { ?answer <authoredBy> <liu> }
}
```

Qwen V replaces the UNION with a conjunction:

```
SELECT DISTINCT ?answer WHERE {
    ?answer <authoredBy> <eppstein> .
    ?answer <authoredBy> <liu>
}
```

This query asks for papers authored by both authors at the same time and returns no rows, since no paper is co-authored by both. Qwen VI keeps the UNION and returns the correct rows.

NEGATION on Qwen also reaches a good score with FQT (+13.5%). When NEGATION examples are mixed with other types in the retrieval, the model sometimes drops the FILTER NOT EXISTS clause; restricting the retrieval to NEGATION keeps the filter in every example.

Three query types, instead, lose a few points on Qwen: DOUBLE_INTENT (-5.5%), MULTI_FACT (-2.1%), and SUPERLATIVE+COMPARATIVE (-2.4%). For these types, the cross-type examples retrieved without FQT were close enough in structure to be useful, and removing them slightly hurts performance.

On Gemini, the effects are much smaller in both directions. The model picks the correct structure even when the retrieved examples are mixed across types. The two regressions, COUNT (-1.5%) and UNION (-0.6%), are too small to be significant. Overall, FQT brings small adjustments rather than large changes on Gemini.

4.4.2.4 Effect of Chain-of-Thought

Four pairs of configurations isolate the effect of CoT: III against IV (with SFS), I against II (system prompt only), V against VII (with DFS), and VI against VIII (with DFS and FQT). The table below reports the macro F1 of each side of the pair and the change introduced by CoT.

Pair	Base prompt	Gem F1	Δ Gem	Qwen F1	Δ Qwen
III → IV	SP + SFS	0.757 → 0.729	-2.8%	0.331 → 0.492	+16.1%
I → II	SP	0.675 → 0.653	-2.2%	0.288 → 0.392	+10.4%
V → VII	SP + DFS	0.898 → 0.902	+0.4%	0.726 → 0.809	+8.3%
VI → VIII	SP + DFS + FQT	0.915 → 0.922	+0.7%	0.790 → 0.813	+2.3%

Table 4.7: *Macro F1 of the base and CoT-active configurations on the four pairs, with the change in percentage points. Rows are sorted by Δ Qwen.*

Qwen always benefits from CoT. Gemini shows a small loss without DFS (I → II, III → IV) and a small gain with DFS (V → VII, VI → VIII). The aggregate gain hides different effects across query types.

Configuration VII combines DFS and CoT. The table below shows the per-type macro F1 for V → VII on both LLMs.

Query type	GemV	GemVII	Δ Gem	QwenV	QwenVII	Δ Qwen
Double_negation	0.910	0.955	+4.5%	0.530	0.765	+23.5%
Union	0.805	0.804	-0.1%	0.478	0.667	+18.9%
Superl.+Comp.	0.832	0.892	+6.0%	0.653	0.725	+7.2%
Disambiguation	0.774	0.862	+8.8%	0.681	0.743	+6.2%
Multi_fact	0.914	0.914	0.0%	0.776	0.833	+5.7%
Count	0.890	0.895	+0.5%	0.675	0.730	+5.5%
Double_intent	0.972	0.993	+2.1%	0.814	0.865	+5.1%
Boolean	0.950	0.940	-1.0%	0.835	0.880	+4.5%
Negation	0.940	0.775	-16.5%	0.865	0.900	+3.5%
Single_fact	0.990	0.995	+0.5%	0.955	0.980	+2.5%
TOTAL	0.898	0.902	+0.4%	0.726	0.809	+8.3%

Table 4.8: *Per-query-type macro F1 for V against VII on both LLMs. The Δ columns show the change in macro F1 introduced by CoT. Rows are sorted by Δ Qwen.*

On Qwen, CoT helps every query type. The largest gains are on DOUBLE_NEGATION (+23.5%) and UNION (+18.9%), the same two types where FQT had the largest gains in the previous section.

For Gemini, V → VII brings only +0.4% overall. CoT gives clear gains on DISAMBIGUATION (+8.8%) and SUPERLATIVE+COMPARATIVE (+6.0%), but a large loss on NEGATION (-16.5%) almost cancels them in the total score.

Configuration VIII combines DFS, FQT, and CoT. The table below shows the per-type macro F1 for VI → VIII on both LLMs.

Query type	GemVI	GemVIII	Δ Gem	QwenVI	QwenVIII	Δ Qwen
Superl.+Comp.	0.836	0.910	+7.4%	0.629	0.729	+10.0%
Boolean	0.955	0.950	-0.5%	0.820	0.900	+8.0%
Disambiguation	0.793	0.899	+10.6%	0.723	0.795	+7.2%
Multi_fact	0.930	0.885	-4.5%	0.755	0.796	+4.1%
Union	0.799	0.811	+1.2%	0.707	0.727	+2.0%
Double_intent	0.979	0.993	+1.4%	0.759	0.766	+0.7%
Count	0.875	0.890	+1.5%	0.680	0.685	+0.5%
Single_fact	0.995	1.000	+0.5%	0.985	0.985	0.0%

Query type	GemVI	GemVIII	Δ Gem	QwenVI	QwenVIII	Δ Qwen
Negation	1.000	0.910	-9.0%	1.000	0.980	-2.0%
Double_negation	0.985	0.975	-1.0%	0.845	0.765	-8.0%
TOTAL	0.915	0.922	+0.7%	0.790	0.813	+2.3%

Table 4.9: *Per-query-type macro F1 for VI against VIII on both LLMs. The Δ columns show the change in macro F1 introduced by CoT. Rows are sorted by Δ Qwen.*

Qwen's largest gains are on three types: SUPERLATIVE+COMPARATIVE (+10.0%), BOOLEAN (+8.0%), and DISAMBIGUATION (+7.2%). FQT had little effect on these three types in the previous section.

A SUPERLATIVE+COMPARATIVE question in DBLP-QuAD asks for a maximum or a minimum, like the year with the most papers or the most frequent co-author. The gold query usually computes a COUNT, groups by the answer variable, and orders the groups by the count. Without CoT, Qwen often skips the COUNT and orders the results by the wrong column. Consider Q0964, "Mention the year in which Koike, Yasuharu published the most papers.". The gold query is

```
SELECT ?answer (COUNT(?answer) AS ?count) WHERE {
    ?x <authoredBy> <author> .
    ?x <yearOfPublication> ?answer
} GROUP BY ?answer ORDER BY DESC(?count) LIMIT 1
```

Qwen VI drops the count and orders by year:

```
SELECT ?answer WHERE {
    ?x <authoredBy> <author> .
    ?x <yearOfPublication> ?answer
} GROUP BY ?answer ORDER BY DESC(?answer) LIMIT 1
```

This query returns the most recent year (2022) instead of the year with the most papers (2021). With CoT, Qwen VIII writes the following reasoning before the query:

```
"""
1. Query type: SUPERLATIVE+COMPARATIVE. The goal is to find the year (?answer)
   ↪ with the highest number of papers.
2. Identify publications authored by `<author>` and their publication years:
   ↪ `?x --authoredBy--> <author>` and `?x --yearOfPublication--> ?answer`.
3. `GROUP BY ?answer` and `COUNT(?answer)` calculates the total papers per
   ↪ year.
4. `ORDER BY DESC(?count) LIMIT 1` selects the year with the maximum count.
"""
```

The reasoning makes the `COUNT` and the `ORDER BY DESC(?count)` explicit. Qwen VIII then writes the gold query and returns the correct year.

BOOLEAN questions ask whether a fact is true or false, and the gold query is a plain ASK on the triples that represent the fact. Without CoT, Qwen often writes a query that does not actually check the fact. Consider Q0315, "Was the paper 'Evaluation of multiresolution image fusion algorithms' co-authored by V. J. D. Tsai and Frederic T.?". The gold query checks that both authors are linked to the paper:

```
ASK {
    <paper> <authoredBy> <author1> .
    <paper> <authoredBy> <author2>
}
```

Qwen VI writes a different query with two FILTER NOT EXISTS clauses:

```
ASK {
    <paper> <authoredBy> ?x .
    FILTER NOT EXISTS { ?x <authoredBy> <author1> }
    FILTER NOT EXISTS { ?x <authoredBy> <author2> }
}
```

The two FILTER NOT EXISTS clauses are always true, because `?x` is a person and `authoredBy` connects a publication to a person, so triples like `?x <authoredBy> <author1>` cannot exist in the graph. The query reduces to "does the paper have any author?", which is true regardless of who the actual authors are. Qwen returns True, but the gold answer is False because Frederic T. is not an author of the paper. With CoT, Qwen VIII writes the following reasoning before the query:

```
"""
1. Identify the query type as BOOLEAN (Yes/No question), requiring an ASK
   ↪ statement.
2. Define the first triple pattern to verify that the publication '<paper>'
   ↪ was authored by the first person '<author1>'.
3. Define the second triple pattern to verify that the same publication was
   ↪ also authored by the second person '<author2>'.
4. Both conditions must be true simultaneously for the query to return true,
   ↪ confirming co-authorship.
"""
```

The reasoning makes both authorship checks explicit. Qwen VIII then writes the gold query and returns the correct answer.

On the types where FQT was already strong (DOUBLE_NEGATION, NEGATION, UNION), CoT adds little and can hurt: DOUBLE_NEGATION drops -8.0%, NEGATION drops -2.0%, and UNION gains only +2.0%.

For Gemini, VI → VIII brings only +0.7% overall. CoT gives clear gains on DISAMBIGUATION (+10.6%) and SUPERLATIVE+COMPARATIVE (+7.4%), but losses on NEGATION (-9.0%) and MULTI_FACT (-4.5%) almost cancel them in the total score. We note that the Gemini API was queried at the minimal thinking level[4], which the documentation describes as the "no thinking" setting for most queries but does not guarantee thinking is disabled.

4.4.2.5 Effect of Static Few-Shot

Three pairs of configurations isolate the effect of SFS: II (SP + CoT) against IV (SP + SFS + CoT), I (SP) against III (SP + SFS), and VIII (SP + DFS + FQT + CoT) against IX (SP + SFS + DFS + FQT + CoT). The table below reports the macro F1 of each side of the pair and the change introduced by SFS.

Pair	Base prompt	Gem F1	Δ Gem	Qwen F1	Δ Qwen
II → IV	SP + CoT	0.653 → 0.729	+7.6%	0.392 → 0.492	+10.0%
I → III	SP	0.675 → 0.757	+8.2%	0.288 → 0.331	+4.3%
VIII → IX	SP + DFS + FQT + CoT	0.922 → 0.927	+0.5%	0.813 → 0.837	+2.4%

Table 4.10: *Macro F1 of the base and SFS-active configurations on the three pairs, with the change in percentage points. Rows are sorted by Δ Qwen.*

SFS gives clear gains when DFS is not in the prompt (I → III and II → IV) and only marginal gains once DFS is in (VIII → IX). DFS is much more effective than SFS as a source of examples: the best SFS-only configuration, IV (SP + SFS + CoT), reaches 0.729 on Gemini and 0.492 on Qwen, well below the simplest DFS-only configuration V (SP + DFS) at 0.898 and 0.726. Adding the 10 static examples on top of DFS (VIII → IX) is also less effective than raising the number of dynamic examples in the same setup, as shown in the next subsection.

[4]https://ai.google.dev/gemini-api/docs/thinking#thinking-levels

4.4.2.6 Number of Dynamic Few-Shot Examples

Configuration VIII is run at four values of k (3, 5, 10, 15) on both LLMs. The table below reports the macro F1 at each value and the change between consecutive values.

k	Gem F1	Δ Gem	Qwen F1	Δ Qwen
3	0.894	–	0.794	–
5	0.922	+2.8%	0.813	+1.9%
10	0.934	+1.2%	0.843	+3.0%
15	0.932	-0.2%	0.852	+0.9%

Table 4.11: *Macro F1 of configuration VIII at four values of k on Gemini and Qwen, with the incremental change between consecutive values.*

On Gemini, macro F1 grows from k = 3 to k = 10. The change between k = 10 and k = 15 is too small to be significant. On Qwen, macro F1 keeps growing through k = 15, with smaller gains at each step.

The smaller model benefits more from a larger k. The aggregate gain hides different effects across query types. The table below shows the per-query-type macro F1 at k = 5 against k = 15 on both LLMs.

Query type	Gem@5	Gem@15	Δ Gem	Qwen@5	Qwen@15	Δ Qwen
Disambiguation	0.899	0.936	+3.7%	0.795	0.935	+14.0%
Double_intent	0.993	1.000	+0.7%	0.766	0.845	+7.9%
Superl.+Comp.	0.910	0.898	-1.2%	0.729	0.774	+4.5%
Count	0.890	0.920	+3.0%	0.685	0.730	+4.5%
Multi_fact	0.885	0.895	+1.0%	0.796	0.820	+2.4%
Union	0.811	0.809	-0.2%	0.727	0.744	+1.7%
Negation	0.910	0.935	+2.5%	0.980	0.995	+1.5%
Boolean	0.950	0.955	+0.5%	0.900	0.910	+1.0%
Double_negation	0.975	0.980	+0.5%	0.765	0.775	+1.0%
Single_fact	1.000	0.995	-0.5%	0.985	0.990	+0.5%
TOTAL	0.922	0.932	+1.0%	0.813	0.852	+3.9%

Table 4.12: *Per-query-type macro F1 at k = 5 against k = 15 on both LLMs. The Δ columns show the change in macro F1. Rows are sorted by Δ Qwen.*

Most of the aggregate gain on Qwen comes from two query types: DISAMBIGUATION (+14.0%) and DOUBLE_INTENT (+7.9%). With more retrieved examples, Qwen sees more variants of the same pattern, as the two cases below illustrate.

DISAMBIGUATION questions ask for a specific entity, identified by a hint such as an author's surname or a paper topic. The hint is used by the entity linker to pick the right URI; it does not appear in the gold SPARQL query, which simply returns the entities related to that URI (for example, all the authors of the linked paper). Consider Q0703, "Which author published the paper on Transportation performance and has the name Apichit?". The gold query is

```
SELECT DISTINCT ?answer WHERE {
    <paper> <authoredBy> ?answer
}
```

Qwen at k = 5 turns the surname into a filter on the answer:

```
SELECT DISTINCT ?answer WHERE {
    <paper> <authoredBy> ?answer
    FILTER(STRSTARTS(?answer, "Apichit"))
}
```

Author URIs in dblp are short identifiers like `<https://dblp.org/pid/ ... >` and not strings, so the `STRSTARTS` filter cannot be applied to them and the query returns no result. Qwen at k = 15 sees more DISAMBIGUATION examples without the filter, and produces the gold query.

DOUBLE_INTENT questions ask for two related pieces of information about the same subject (e.g., papers and their year, authors and their affiliations). The gold query uses one triple for each piece. Consider Q0221, "List the papers published by Simone D. and in which year.". The gold query is

```
SELECT DISTINCT ?firstanswer ?secondanswer WHERE {
    ?firstanswer <authoredBy> <author> .
    ?firstanswer <yearOfPublication> ?secondanswer
}
```

Qwen at k = 5 adds an extra triple `?x <title> ?firstanswer` that binds `?firstanswer` to the paper title instead of the paper URI:

```
SELECT DISTINCT ?firstanswer ?secondanswer WHERE {
    ?x <authoredBy> <author> .
    ?x <title> ?firstanswer .
    ?x <yearOfPublication> ?secondanswer
}
```

This query returns title strings instead of paper URIs, so the answer set does not match the gold one. Qwen at k = 15 sees more DOUBLE_INTENT examples that return paper URIs, and produces the gold query.

For Gemini, k = 5 against k = 15 brings only +1.0% overall. The largest gains are on DISAMBIGUATION (+3.7%) and COUNT (+3.0%), and three types regress slightly (SUPERLATIVE+COMPARATIVE -1.2%, SINGLE_FACT -0.5%, UNION -0.2%). Gemini already reaches high scores at k = 5, and additional examples bring smaller gains.

The best configurations from the Query Generator ablation are reported in the table below. They are used as the baseline for the Relation Linker ablation in the next section.

LLM	Configuration	macro F1
Gemini	VIII at k = 10	0.934
Qwen	VIII at k = 15	0.852

Table 4.13: *Best Query Generator configurations on each LLM.*

4.4.3 Relation Linker Ablation

4.4.3.1 Aggregate Results

The two tables below report the aggregate metrics of the relation linker ablation on Gemini and on Qwen. The first row of each table is the full-schema baseline (the best Query Generator configuration of table 4.13, with the relation linker held at the full schema). The other rows are the RAG runs of X (without domain and range) at 10, 15, and 20 retrieved properties, and of XI (with domain and range) at 10 and 20.

Linker	Properties	EM	macro F1	empty	micro F1	DBLP F1
Full schema	–	91.5%	0.934	35	0.990	0.933
X	10	89.3%	0.913	47	0.982	0.912
X	15	90.9%	0.927	38	0.989	0.925
X	20	90.4%	0.923	35	0.989	0.920
XI	10	88.5%	0.906	57	0.978	0.904
XI	20	90.1%	0.920	38	0.988	0.919

Table 4.14: *Aggregate metrics on the test split for the relation linker ablation on Gemini.*

Linker	Properties	EM	macro F1	empty	micro F1	DBLP F1
Full schema	–	83.5%	0.852	128	0.944	0.848
X	10	82.0%	0.837	179	0.938	0.832
X	15	82.3%	0.842	150	0.955	0.840
X	20	83.9%	0.856	150	0.970	0.851
XI	10	82.0%	0.839	165	0.958	0.836
XI	20	84.1%	0.859	130	0.961	0.854

Table 4.15: *Aggregate metrics on the test split for the relation linker ablation on Qwen.*

On Qwen, two RAG runs slightly exceed the full-schema baseline of 0.852: XI at 20 retrieved properties (0.859, +0.7%) and X at 20 (0.856, +0.4%). The runs at 10 and 15 properties stay below.

On Gemini, the full schema stays on top at 0.934, and no RAG configuration reaches it. The closest run is X at 15 properties (0.927, -0.7%).

The empty count of the RAG runs is higher than the baseline at 10 retrieved properties (179 on Qwen X, 57 on Gemini XI), and returns close to the baseline at 20 properties (130 on Qwen XI, 35 on Gemini X).

4.4.3.2 Effect of the Number of Retrieved Properties

The number of retrieved properties is the dominant variable of the RAG relation linker. On every configuration, the macro F1 grows monotonically with this number: on Qwen, X moves from 0.837 at 10 properties to 0.856 at 20 (+1.9%) and XI from 0.839 to 0.859 (+2.0%); on Gemini, X moves from 0.913 to 0.923 (+1.0%) and XI from 0.906 to 0.920 (+1.4%).

At 20 retrieved properties, the relations block of the prompt lists 20 predicates instead of the 75 of the full schema, a reduction of about 73% in the number of predicates exposed to the LLM. The smaller model benefits from this focused input and reaches a macro F1 above the full-schema baseline, while the larger one performs better with the full schema.

On Qwen, the effect of moving from 10 to 20 retrieved properties is illustrated below on a MULTI_FACT question. This query type requires connecting two or more facts to answer, so the gold query uses one predicate for each fact. When k is small, one of these predicates may not appear in the retrieved list. Consider Q0120, "Which venues has A. J. Biega published in?", whose gold query uses `authoredBy` and `publishedIn`:

```
SELECT DISTINCT ?answer WHERE {
    ?x <authoredBy> <author> .
    ?x <publishedIn> ?answer
}
```

At 10 retrieved properties, `authoredBy` is not among the predicates returned by the relation linker; the only retrieved predicate that connects a person to a publication is `editedBy`. The model uses it as a stand-in:

```
SELECT DISTINCT ?answer WHERE {
    <author> <editedBy> ?x .
    ?x <publishedIn> ?answer
}
```

The query returns no rows: in DBLP, `editedBy` connects a publication to its editor, so the triple pattern `<author> <editedBy> ?x` does not match anything in the graph. At 20 retrieved properties, `authoredBy` is included in the list returned by the relation linker, and the model produces the gold query, which returns the 18 venues in which the author has published.

4.4.3.3 Effect of the Domain-Range Information

Adding the domain and range fields to each retrieved property (XI against X) changes the macro F1 by -0.3% on Gemini at 20 retrieved properties (0.923 to 0.920) and by +0.3% on Qwen (0.856 to 0.859); at 10 retrieved properties, the changes are -0.7% on Gemini and +0.2% on Qwen. The macro F1 changes very little on the four pairs, and the domain and range information has no measurable effect.

4.4.4 Gemini vs Qwen

The two best configurations identified in the previous sections, Gemini VIII at k = 10 with the full schema and Qwen X at k = 15 with 20 retrieved properties, are compared per query type in the table below.

Query type	Gem VIII@10	Qwen X@15 RRR=20	Δ (Gem − Qwen)
Count	0.915	0.685	+23.0%
Multi_fact	0.925	0.755	+17.0%
Double_negation	0.975	0.835	+14.0%
Superl.+Comp.	0.900	0.772	+12.8%
Double_intent	1.000	0.899	+10.1%
Union	0.807	0.759	+4.8%
Disambiguation	0.941	0.925	+1.6%
Boolean	0.955	0.950	+0.5%
Single_fact	0.995	0.990	+0.5%
Negation	0.930	0.990	-6.0%
TOTAL	0.934	0.856	+7.8%

Table 4.16: *Macro F1 of the best Gemini and best Qwen configurations on each query type. The Δ column shows the difference Gemini − Qwen. Rows are sorted by Δ in decreasing order.*

The aggregate gap between the two LLMs is +7.8%. The gap is +10% or more on five query types (COUNT, MULTI_FACT, DOUBLE_NEGATION, SUPERLATIVE+COMPARATIVE, DOUBLE_INTENT), with the largest gap on COUNT (+23.0%). It stays below +5% on four other types (UNION, DISAMBIGUATION, BOOLEAN, SINGLE_FACT). NEGATION is the only query type where Qwen exceeds Gemini, by 6.0%.

The two configurations also differ in how often they fail to produce any answer. The best Gemini returns 35 empty answers on the test split, the best Qwen returns 150, about four times more.

4.5 Significance Evaluation of the Obtained Results and Potential Enhancements

4.5.1 Q0: Comparison with the State of the Art

The best Gemini and Qwen configurations score above the three published systems on the comparison metrics. The Qwen result is the more relevant one: the 4B model runs locally and reaches a micro F_1 that is 12.1% higher than NLQxform and a DBLP-QuAD F_1 that is 9.0% higher than ASK-DBLP.

ASK-DBLP is the closest system to ours in design: both prompt an LLM without fine-tuning and retrieve dynamic few-shot examples from the training set. The three other techniques of our pipeline (query-type filtering, Chain-of-Thought, and RAG-based relation linking) are not used in ASK-DBLP, and the 9.0% gap is similar to the gains that these techniques bring in the ablation. The comparison is not strictly controlled, however. ASK-DBLP runs on Qwen 2.5 Coder 32B while our system runs on Qwen 3.5 4B, a more recent but eight times smaller model. Part of the 9.0% gap may therefore come from these model differences, not only from the techniques. Running our pipeline on the same 32B model would isolate the effect of the techniques, but we did not have the resources to do it.

Moreover, our pipeline holds the Information Extractor and the Entity Linker at their gold variant, while NLQxform and ASK-DBLP perform entity linking on their own; our scores are therefore an upper bound on the end-to-end pipeline. NLQxform and T5-Base, on the other hand, are fine-tuned on the DBLP-QuAD training set, and NLQxform also builds a template base from its gold queries. The dataset itself contains only 3.8% of test questions built on templates not seen in training, so the reported scores reflect mostly the performance on familiar patterns.

4.5.2 Q1: Contribution of the Prompting Techniques

Dynamic few-shot is the technique with the largest impact on both LLMs. Without it, the macro F_1 stays below 0.50 on Qwen and below 0.76 on Gemini. With it as the only added technique, the macro F_1 reaches 0.726 on Qwen and 0.898 on Gemini. The other three techniques, static few-shot, query-type filtering, and Chain-of-Thought, are corrections on top of this baseline.

On Qwen, query-type filtering and Chain-of-Thought correct the same class of error: the model confuses query patterns that look similar in natural language but produce different SPARQL structures, like NEGATION and DOUBLE_NEGATION, or UNION and conjunction. The filter prevents the confusion by keeping only dynamic examples of the predicted query type, so the model sees only examples that match the structure it has to produce. Chain-of-Thought does the same in a different way: it asks the model to write the reasoning steps before the query, so the model decides the query type and the target variable before assembling the triples. The two techniques cover the same kind of mistake, so their gains do not fully add up when used together. Once the filter is in the prompt, Chain-of-Thought shifts its contribution to query types that the filter does not protect, like SUPERLATIVE+COMPARATIVE, BOOLEAN, and DISAMBIGUATION. Static few-shot helps Qwen only when dynamic few-shot is absent; once the dynamic block is in the prompt, its contribution is smaller than the gain obtained by retrieving more dynamic examples.

On Gemini, the same three techniques bring only small adjustments. The model produces useful queries already without dynamic few-shot, with macro F_1 above 0.70 from static few-shot alone, and once the dynamic block is in the prompt each of the other techniques (query-type filtering, Chain-of-Thought, and static few-shot) adds less than 2%. The reason is that Gemini does not need help to distinguish similar query patterns or to assemble structured queries: it handles them correctly from the dynamic examples alone, while a smaller model like Qwen needs the extra support.

The number of retrieved dynamic examples follows the same logic. Gemini stops improving at k = 10, while Qwen keeps improving up to k = 15, the largest value tested. The smaller model benefits from seeing more variants of the same pattern; the larger one already generalizes from fewer.

4.5.3 Q2: Dynamic Relation Linking

The full DBLP ontology used by the baseline contains about 75 predicates, and not all of them are relevant to a given question. The RAG relation linker retrieves only the k most similar predicates, which at k = 20 reduces the relations block of the prompt by about 73%. Whether this trade-off pays off depends on the LLM.

On Qwen, RAG matches or slightly exceeds the full schema. With 20 retrieved predicates, the macro F_1 stays at the baseline level or moves above it by less than 1%. The smaller model benefits from a focused list: when the full ontology is in the prompt, the predicates that are not relevant to the question can confuse the model, and removing them helps it pick the right ones more reliably.

On Gemini, the full schema reaches the highest macro F_1, and no RAG run matches it; the best RAG result is about 0.7% below the baseline. The larger model handles the full ontology without trouble, so the small gain observed on Qwen does not appear here, and removing predicates produces a small loss rather than a gain.

The number of retrieved predicates k is the dominant variable of the RAG relation linker: on both LLMs the macro F_1 grows when k moves from 10 to 20, since a larger k is less likely to miss a predicate needed by the gold query. The domain and range information attached to each retrieved predicate, instead, has no measurable effect.

4.5.4 Q3: Gemini vs Qwen

A 4B-parameter quantized model is competitive with Gemini on the same pipeline: the best Qwen reaches a macro F_1 of 0.856 on the test split, against 0.934 for the best Gemini, a gap of 7.8%. The gap is not the same on every query type: it stays small on five types and is larger than 10% on five others.

Four of the five large gaps share a structural cause: COUNT, MULTI_FACT, SUPERLATIVE+COMPARATIVE, and DOUBLE_INTENT. Their gold queries are not a single triple, but combine an aggregation, a chain of triples through a shared variable, an ORDER BY with a LIMIT, or two answer variables. Qwen often gets the structure wrong, for example by skipping an aggregation, adding an unnecessary triple, or ordering by the wrong variable. Gemini produces the right structure more often from the same dynamic examples.

The fifth large gap is on DOUBLE_NEGATION, where the reason is different. The gold SPARQL is a plain ASK on a single triple; the difficulty is that the question wording is easy to confuse with NEGATION, and the smaller model gets this distinction wrong more often than Gemini.

On the simpler types the gap stays close to zero: SINGLE_FACT, BOOLEAN, DISAMBIGUATION, and UNION are all within 5%. Their gold queries are short and follow patterns that

both models reproduce well. NEGATION is the only type where Qwen exceeds Gemini, by 6%. With dynamic few-shot and query-type filtering, the best Qwen produces the FILTER NOT EXISTS pattern almost always (0.990); the best Gemini drops to 0.930 on the same type because Chain-of-Thought, part of the best configuration, hurts on NEGATION.

Another difference is that, even at the best configuration, Qwen fails on some questions by producing a malformed query or no answer at all, rather than a wrong one. This kind of failure happens about four times more often than on Gemini.

4.5.5 Potential Enhancements

The Information Extractor and the Entity Linker are evaluated in this chapter only in their gold variant. The LLM-based and API-based variants of section 3.2.2 and section 3.2.3 were designed for the demo of section 3.2.7, where a human can review and correct the output of each stage, and are not accurate enough to run without supervision. Reaching the quality required by an automatic pipeline is a project of its own, as confirmed by DBLPLink 2.0 [29], which the authors of ASK-DBLP developed as a separate effort. A possible direction is to merge the two modules into a single agent that uses the tool-calling feature of the LLM. The agent receives the question, decides which mentions need a URI, calls the dblp search API for each of them, and retries the call when the result does not match the mention. This design replaces the fixed two-stage flow with a flexible one.

The RAG Relation Linker of section 3.2.4 retrieves the top k predicates by cosine similarity between the user question and the natural-language description of each predicate, taken from the dblp ontology. Two directions could improve this module. The first is to rewrite the predicate descriptions before the offline indexing, since the official descriptions are short and sometimes ambiguous, and a richer description should make the retrieval more reliable. The second is to combine the simple RAG with the structure of the ontology, for example through a Graph RAG approach. The dblp ontology is itself a graph, with a sub-class hierarchy and with domain and range edges between predicates and classes. A retrieval step that follows these connections could find predicates that the description-based search misses. Some variants in this direction were explored during development. One variant looked at the parent-child relationships among the retrieved predicates: when a parent and one of its children were both retrieved, only the one with the higher score was kept. Another variant filtered out predicates whose domain or range did

not match the types of the named entities in the question, although this rule rejected only a few predicates in practice. Due to time constraints, none of these variants was tuned enough to be included in the final ablation.

Two enhancements could be added to the Query Generator. The first is self-verification: after producing the SPARQL query, the LLM is asked to review it and to correct it if it spots a mistake. The second is to revisit the use of structured output on Qwen. The current pipeline asks Qwen to produce the reasoning and the SPARQL query together, as a single structured object. A possible alternative is to split the generation into two calls. In the first call, the model produces the reasoning and the query as free text. In the second call, the model is given its own previous output and is asked to extract the query into a JSON field. The hypothesis is that the smaller model handles the constrained generation better when the schema is simple.

Both the RAG Relation Linker and the dynamic few-shot retrieval use `gemini-embedding-001`, which is a cloud embedding model. No other embedding model has been evaluated, due to time constraints. An open direction is to repeat the experiments with an embedding model that runs locally on the same hardware as Qwen, for example Qwen3-Embedding, and to compare the retrieval quality and the effect on the downstream pipeline.

The current pipeline assumes that the user question is well-formed and that the generated SPARQL query is correct on the first attempt. Three robustness mechanisms would improve the behaviour of the system in real use. The first is an input check at the start of the pipeline, where an LLM-based component detects whether the question is vague or out of scope and asks the user to reformulate it. ASK-DBLP [28] uses the same mechanism. The second is a self-correction loop on the SPARQL query. When the query fails to parse or returns an empty result, the system sends the query, the error, and the original question back to the LLM, and asks for a new query. The loop runs until the query is valid or a maximum number of iterations is reached. Frameworks such as LangGraph[5] make this loop simple to implement. The third is a fallback when the self-correction loop does not converge: the system can either ask the user to reformulate the question, or, when the local LLM is in use, fall back to the cloud LLM. The fallback option is most relevant for Qwen, which at the best configuration produces about four times more empty answers than Gemini.

DBLP-QuAD is built from 98 hand-written SPARQL templates with placeholders filled from

[5] https://www.langchain.com/langgraph

the dblp KG. The dataset covers a fixed set of structural patterns and may not reflect the variety of questions that real users would ask. The authors of DBLP-QuAD have released a follow-up dataset, DBLP-QuAD 2.0 [30], with a different construction. The SPARQL queries are taken from the query logs of the public dblp endpoint, and the natural-language question for each query is generated by an LLM, with the questions of the test set verified manually. The dataset reflects user information needs more closely than DBLP-QuAD, since the SPARQL queries come from real users, and it also covers the recent extensions of the dblp schema, such as the introduction of the `publishedInStream` predicate. Running the same pipeline and ablation on DBLP-QuAD 2.0 would test whether the conclusions of section 4.4 still hold on these more realistic questions.

Bibliography

[1] M. R. Ackermann, H. Bast, B. M. Beckermann, J. Kalmbach, P. Neises, and S. Ollinger, "The dblp Knowledge Graph and SPARQL Endpoint," *Transactions on Graph Data and Knowledge*, vol. 2, no. 2, pp. 3:1–3:23, 2024, doi: 10.4230/TGDK.2.2.3.

[2] D. Banerjee, P. A. Nair, J. N. Kaur, R. Usbeck, and C. Biemann, "Modern Baselines for SPARQL Semantic Parsing," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, in SIGIR '22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 2260–2265. doi: 10.1145/3477495.3531841.

[3] D. Banerjee, S. Awale, R. Usbeck, and C. Biemann, "DBLP QuAD: A Question Answering Dataset over the DBLP Scholarly Knowledge Graph." arXiv, Mar. 2023. doi: 10.48550/arXiv.2303.13351.

[4] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Scientific American*, vol. 284, no. 5, pp. 34–43, May 2001, doi: 10.1038/scientificamerican0501-34.

[5] RDF Working Group, "RDF 1.1." Feb. 2014. Available: https://www.w3.org/TR/rdf11-concepts/

[6] RDF Working Group, "RDF Schema 1.1." Feb. 2014. Available: https://www.w3.org/TR/rdf11-schema/

[7] R. Studer, V. R. Benjamins, and D. Fensel, "Knowledge engineering: Principles and methods," *Data & Knowledge Engineering*, vol. 25, no. 1, pp. 161–197, Mar. 1998, doi: 10.1016/S0169-023X(97)00056-6.

[8] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, Jun. 1993, doi: 10.1006/knac.1993.1008.

[9] W. N. Borst, "Construction of engineering ontologies for knowledge sharing and reuse," PhD thesis, 1997. doi: 10.3990/1.9789036509886.

[10] OWL Working Group, "OWL." Feb. 2004. Available: https://www.w3.org/TR/owl-features/

[11] SPARQL Working Group, "SPARQL 1.2 Query Language." Mar. 2026. Available: https://www.w3.org/TR/sparql12-query/#abstract

[12] T. Berners-Lee, “Linked Data.” Jul. 2006. Available: https://www.w3.org/DesignIssues/LinkedData.html

[13] S. Peroni and D. Shotton, “The SPAR Ontologies,” in *The Semantic Web – ISWC 2018*, vol. 11137, D. Vrandečić, K. Bontcheva, M. C. Suárez-Figueroa, V. Presutti, I. Celino, M. Sabou, L.-A. Kaffee, and E. Simperl, Eds., Cham: Springer International Publishing, 2018, pp. 119–136. doi: 10.1007/978-3-030-00668-6_8.

[14] A. Hogan *et al.*, “Knowledge Graphs,” *ACM Comput. Surv.*, vol. 54, no. 4, pp. 71:1–71:37, Jul. 2021, doi: 10.1145/3447772.

[15] B. D’Arcus and F. Giasson, “BIBO,” *DCMI*. May 2016. Accessed: Mar. 28, 2026. [Online]. Available: https://www.dublincore.org/specifications/bibo/

[16] DCMI Usage Board, “DCMI Metadata Terms,” *DCMI*. Jan. 2020. Available: https://www.dublincore.org/specifications/dublin-core/dcmi-terms/

[17] S. Peroni and D. Shotton, “FaBiO and CiTO: Ontologies for describing bibliographic resources and citations,” *Journal of Web Semantics*, vol. 17, pp. 33–43, Dec. 2012, doi: 10.1016/j.websem.2012.08.001.

[18] S. Peroni and D. Shotton, “The DataCite Ontology.” Sep. 2022. Accessed: Mar. 30, 2026. [Online]. Available: https://sparontologies.github.io/datacite/current/datacite.html

[19] S. Peroni and D. Shotton, “OpenCitations, an infrastructure organization for open scholarship,” *Quantitative Science Studies*, vol. 1, no. 1, pp. 428–444, Feb. 2020, doi: 10.1162/qss_a_00023.

[20] Wikidata contributors, “Wikidata.” 2026. Available: https://www.wikidata.org/wiki/Wikidata:Main_Page

[21] T. A. Taffa and R. Usbeck, “Leveraging LLMs in Scholarly Knowledge Graph Question Answering.” arXiv, Nov. 2023. doi: 10.48550/arXiv.2311.09841.

[22] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” *arXiv.org*. May 2020. doi: 10.48550/arXiv.2005.14165.

[23] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv.org*. May 2020. doi: 10.48550/arXiv.2005.11401.

[24] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv.org*. Jan. 2022. doi: 10.48550/arXiv.2201.11903.

[25] R. Usbeck *et al.*, “QALD-10 – The 10th challenge on question answering over linked data,” *Semantic Web*, pp. 1–15, Nov. 2023, doi: 10.3233/SW-233471.

[26] S. Auer *et al.*, “The SciQA Scientific Question Answering Benchmark for Scholarly Knowledge,” *Scientific Reports*, vol. 13, no. 1, p. 7240, May 2023, doi: 10.1038/s41598-023-33607-z.

[27] R. Wang, Z. Zhang, L. Rossetto, F. Ruosch, and A. Bernstein, “NLQxform: A Language Model-based Question to SPARQL Transformer.” arXiv, Nov. 2023. doi: 10.48550/arXiv.2311.07588.

[28] T. Taffa *et al.*, “ASK-DBLP: Answering Questions over DBLP ⋆,” Nov. 2025. Available: https://www.researchgate.net/publication/396447741_ASK-DBLP_Answering_Questions_over_DBLP

[29] D. Banerjee, T. A. Taffa, and R. Usbeck, “DBLPLink 2.0 – An Entity Linker for the DBLP Scholarly Knowledge Graph.” arXiv, Oct. 2025. doi: 10.48550/arXiv.2507.22811.

[30] T. A. Taffa *et al.*, “DBLP QuAD 2.0: Scholarly Natural Questions from SPARQL,” in *Proceedings of the 13th Knowledge Capture Conference 2025*, in K-CAP ’25. New York, NY, USA: Association for Computing Machinery, Dec. 2025, pp. 236–240. doi: 10.1145/3731443.3771376.